Plataforma de simulación y visualización de modelos de difusión del óxido nítrico

Andrea Santana López

Grado en Ingeniería Informática

Universidad de Las Palmas de Gran Canaria

2026

TFT04

Portada en gris

AGRADECIMIENTOS

Quiero expresar mi más sincero agradecimiento a mi familia por su apoyo financiero, que ha sido fundamental para culminar mis estudios. A mi mejor amiga Yamiley, por brindarme apoyo emocional en los momentos más difíciles. A mi esposo Alejandro, por su inestimable ayuda en los aspectos mentales, financieros y de cuidado durante toda mi formación universitaria. Sin su respaldo, este logro no habría sido posible.

RESUMEN

El presente Trabajo de Fin de Grado consiste en el desarrollo de una plataforma web destinada a investigadores y estudiantes de neurociencia computacional. Esta herramienta permite probar modelos de inteligencia artificial con el objetivo de avanzar en sus investigaciones o proyectos. La plataforma ofrece funcionalidades para el almacenamiento y la gestión de modelos, así como para la visualización y el análisis de datos científicos, tanto de forma estática como dinámica. Además, permite la exportación de gráficas generadas a partir de los resultados obtenidos durante las simulaciones.

Gracias a esta plataforma, los usuarios podrán evaluar, analizar y comparar el comportamiento de diferentes modelos de manera eficiente. Esto facilita la interpretación de los datos y la obtención de conclusiones que contribuyan al desarrollo de nuevas investigaciones y proyectos en el ámbito de la neurociencia computacional.

ABSTRACT

This Bachelor's Thesis presents the development of a web platform designed to assist researchers and students in computational neuroscience by enabling them to test their artificial intelligence models. The platform provides tools for model storage and management, as well as for the visualization and analysis of scientific data in both static and dynamic forms. Additionally, it allows the export of graphs generated from the results obtained during simulations.

Through this platform, users can efficiently evaluate, analyze, and compare the behavior of different models, facilitating data interpretation and the extraction of meaningful conclusions. This contributes to the development of new research and projects within the field of computational neuroscience.

Índice

1. INTRODUCCIÓN **8**
1.1. Estructura del documento . . . 9
1.2. Estado actual y objetivos iniciales . . . 9
1.2.1. Motivación y antecedentes . . . 9
1.3. Objetivos . . . 9
1.3.1. Objetivos principales . . . 9
1.3.2. Objetivos secundarios . . . 9
1.4. Aportaciones del trabajo . . . 10
1.4.1. Principales aportaciones . . . 10
1.4.2. Competencias específicas . . . 10
1.5. Alineamiento con los objetivos de desarrollo sostenible . . . 11

2. Planificación y Metodología **12**
2.1. Roles del desarrollador del proyecto . . . 12
2.2. Herramientas usadas para la planificación . . . 13
2.3. Planificación Temporal con Diagrama de Gantt . . . 14
2.4. Resumen del Product Backlog . . . 14
2.5. Gráficas de Seguimiento del Proyecto . . . 16
2.5.1. Gráfica Burndown . . . 16
2.5.2. Gráfica Burnup . . . 16

3. Desarrollo **17**
3.1. Tecnologías empleadas . . . 17
3.2. Dependencias del proyecto . . . 19
3.3. Entorno de desarrollo . . . 20
3.4. Entorno de pruebas y validación . . . 21
3.5. Estructura del proyecto . . . 21

4. Diseño **23**
4.1. Características de la interfaz de usuario (UI) . . . 23
4.2. Patrones de diseño y decisiones . . . 26
4.2.1. Patrones de diseño . . . 26
4.2.2. Justificación de los patrones de diseño . . . 26
4.3. Prototipos y Mockups . . . 27
4.4. Accesibilidad . . . 40
4.4.1. Ideas de accesibilidad de la plataforma web . . . 40
4.4.2. Integraciones de accesibilidad a la plataforma web . . . 41

5. Modelos **41**
5.1. Diagrama E/R . . . 41

6. Arquitectura **41**
6.1. Arquitectura del sistema . . . 41
6.2. Flujo funcional de la aplicación . . . 42
6.3. Configuración de Docker para la base de datos . . . 43
6.4. Configuración de Docker para la aplicación . . . 45
6.5. Seguridad de la Arquitectura . . . 46
6.5.1. Ideas de mejora de la seguridad . . . 46
6.5.2. Investigación para mejorar la seguridad de la plataforma web . . . 46
6.5.3. Integraciones de la seguridad de la plataforma web . . . 46

7. Desarrollo del simulador de los modelos de difusión del NO **47**

8. Resultados del simulador **60**

9. Conclusiones y trabajo futuro **62**
9.1. Conclusiones . . . 62
9.2. Trabajo futuro . . . 62

10. BIOGRAFÍA **63**

11. Apéndice A: Abreviaturas **66**

12. Apéndice B: Uso de la IA **68**

13. Apéndice C: Estado del arte del modelado del óxido nítrico **69**
13.1. Evolución de los modelos (1994-2026) . . . 69
13.2. Modelos clave y sus aportaciones . . . 69
13.2.1. Modelos analíticos (1994-1998) . . . 69
13.2.2. Modelos anatómicos (2000-2005) . . . 69
13.2.3. Modelos compartimentales (2001-2011) . . . 70
13.2.4. Modelo ANDINO: redes de autómatas (2011) . . . 70
13.2.5. Modelos de difusión anómala y orden fraccional (2024-2025) . . . 70
13.2.6. Simuladores integrados: NODS (2024-2025) . . . 70
13.3. Tendencias actuales y direcciones futuras . . . 71

1. INTRODUCCIÓN

El óxido nítrico (NO) es una molécula gaseosa que desempeña un papel fundamental en numerosos procesos fisiológicos del organismo. Aunque durante décadas se consideró únicamente un subproducto metabólico sin una función biológica relevante, el descubrimiento de su papel como molécula señalizadora supuso un importante avance en la comunidad científica. Este hallazgo fue reconocido en 1998 con la concesión del Premio Nobel de Fisiología o Medicina a Robert F. Furchgott, Louis J. Ignarro y Ferid Murad, quienes demostraron que el óxido nítrico actúa como un mensajero biológico involucrado en múltiples mecanismos de regulación celular.

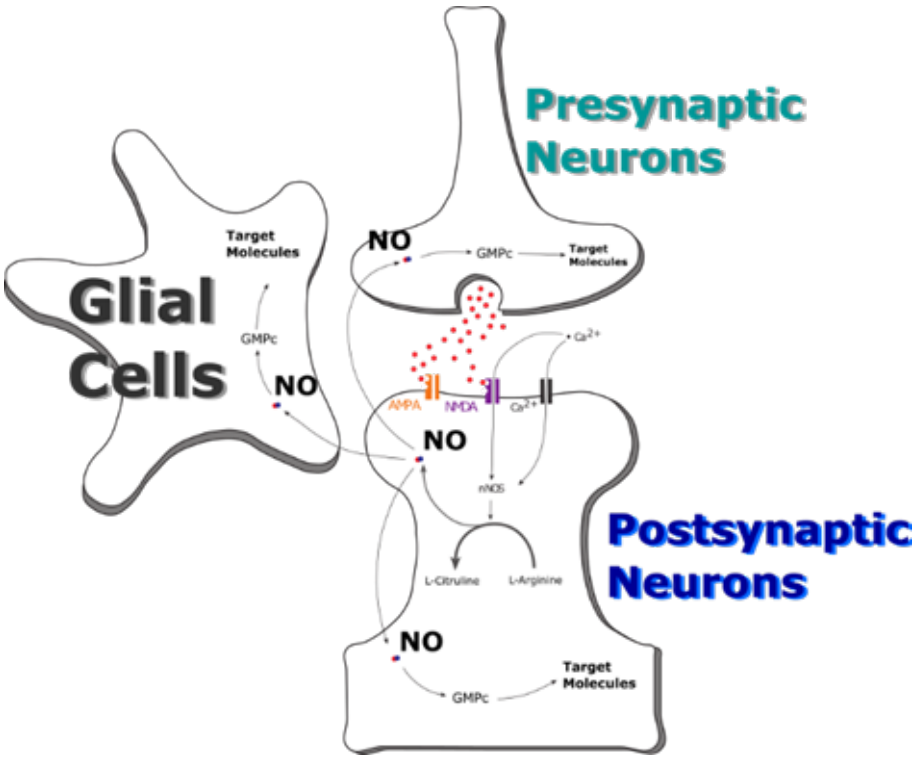


Figura 1 Comunicación celular donde interviene el óxido nítrico

A partir de este descubrimiento, el interés científico por el óxido nítrico experimentó un crecimiento significativo, dando lugar a numerosas investigaciones en áreas como la biología, la fisiología, la química, la farmacología y la medicina. Entre los distintos sistemas biológicos en los que interviene, el sistema nervioso central ha despertado especial interés debido a la participación del NO en procesos relacionados con la neurotransmisión, la plasticidad sináptica y la comunicación neuronal.

El estudio experimental del comportamiento del óxido nítrico presenta importantes desafíos. Su elevada capacidad de difusión, su corta vida media y la complejidad del entorno biológico en el que actúa dificultan la observación directa de su comportamiento espacial y temporal. Por este motivo, numerosos investigadores han recurrido al desarrollo de modelos matemáticos y computacionales que permiten simular los procesos de producción, difusión y degradación del NO bajo diferentes condiciones experimentales.

Estos modelos constituyen una herramienta de gran utilidad para comprender el comportamiento del óxido nítrico y analizar escenarios que resultarían difíciles, costosos o incluso imposibles de estudiar mediante experimentación directa. Sin embargo, la ejecución de dichas simulaciones suele requerir conocimientos técnicos específicos, la instalación de dependencias de software y el uso de herramientas independientes para la visualización de los resultados. Estas limitaciones pueden dificultar tanto la accesibilidad a los modelos como la reproducibilidad de los experimentos realizados.

En este contexto surge la necesidad de disponer de herramientas que faciliten la ejecución y

el análisis de los modelos de difusión del óxido nítrico. Una plataforma web permite centralizar el acceso a diferentes modelos de simulación en un único entorno, accesible desde múltiples dispositivos y sin necesidad de realizar instalaciones locales. Asimismo, la incorporación de sistemas de visualización gráfica facilita la interpretación de los resultados obtenidos y mejora la experiencia de los usuarios durante el proceso de análisis.

El presente Trabajo de Fin de Grado tiene como objetivo el diseño y desarrollo de una plataforma web para la simulación y visualización de modelos de difusión del óxido nítrico en el sistema nervioso central. La plataforma permitirá ejecutar diferentes modelos previamente desarrollados e interpretar sus resultados mediante representaciones gráficas interactivas, estáticas y dinámicas, proporcionando una herramienta accesible y flexible para investigadores y usuarios interesados en el estudio de este neurotransmisor.

La solución propuesta busca mejorar la usabilidad y accesibilidad de los modelos existentes, facilitando su ejecución desde distintos dispositivos y reduciendo la complejidad asociada a la configuración del entorno de trabajo. De esta forma, se pretende proporcionar una herramienta que contribuya tanto a la investigación como a la divulgación y comprensión de los procesos de difusión del óxido nítrico.

1.1. Estructura del documento

La presente memoria se organiza en varios capítulos. En primer lugar, se introduce el contexto y la motivación del trabajo. Posteriormente, se describen los objetivos y requisitos de la aplicación desarrollada. A continuación, se presentan el diseño y la arquitectura de la plataforma web, junto con los modelos de datos utilizados durante su implementación. Seguidamente, se detalla el proceso de desarrollo, las interfaces implementadas y las principales funcionalidades incorporadas. Finalmente, se exponen las conclusiones obtenidas y las posibles líneas de trabajo futuro derivadas de este proyecto.

1.2. Estado actual y objetivos iniciales

1.2.1. Motivación y antecedentes

Actualmente, no existen muchas plataformas que visualicen datos científicos de forma estática y dinámica en el ámbito de la neurociencia computacional.

Por ello, la idea es desarrollar una plataforma extensible donde se puedan analizar los datos que los distintos modelos ejecutan, con el fin de comprobar o comparar su comportamiento. Modelos relacionados con el óxido nítrico o modelos con tipologías similares pueden ayudar a realizar avances en los campos de la tecnología y de la salud, permitiendo comprender cómo funcionan y difundir el conocimiento adquirido por los investigadores. Por esta razón, la creación de esta plataforma busca ayudar a investigadores o estudiantes de neurociencia computacional a probar sus modelos de inteligencia artificial para avanzar en sus investigaciones o proyectos.

1.3. Objetivos

1.3.1. Objetivos principales

- Desarrollar una aplicación web que constituya una plataforma de simulación y visualización de las distintas expresiones para la difusión del NO en sistemas biológicos y artificiales.
- Obtener un producto cuya funcionalidad de simulación permita realizar estudios y análisis acerca de la implicación del gas NO en los procesos de aprendizaje y memoria en el cerebro y en redes neuronales artificiales.

1.3.2. Objetivos secundarios

- Extensible a otros modelos de comportamiento similar.
- Recopilación de distintas investigaciones sobre el óxido nítrico.
- Realización de pruebas funcionales de la adaptabilidad del modelo.

1.4. Aportaciones del trabajo

1.4.1. Principales aportaciones

Gracias a este proyecto, muchos investigadores y estudiantes de neurociencia computacional podrán visualizar el comportamiento de sus modelos de forma eficiente e intuitiva, representando los parámetros de entrada en tablas y los resultados de sus modelos en gráficas que puedan ayudar al campo de la neurociencia a mejorar las investigaciones futuras.

1.4.2. Competencias específicas

Para el desarrollo de este proyecto, se requieren las siguientes competencias técnicas distribuidas en los distintos componentes de la arquitectura:

1. Backend (Django y Python)

- Python scripting: Conocimiento sólido del lenguaje de programación Python, dado que el proyecto se desarrolla sobre el framework Django.
- Protocolo HTTP y modelo cliente-servidor: Comprensión del funcionamiento del protocolo HTTP, ya que Django opera bajo el modelo cliente-servidor para gestionar las comunicaciones entre el usuario y la aplicación.
- Arquitectura MVT (Model-View-Template): Conocimiento del patrón arquitectónico propio de Django, que organiza la aplicación en tres capas diferenciadas (Modelo, Vista y Plantilla).
  - Modelo: Manejo del ORM (Object-Relational Mapper) de Django, que traduce las clases definidas en Python a tablas en la base de datos. Es necesario tener nociones de PostgreSQL para evitar complicaciones durante el desarrollo.
  - Vista: Capacidad para desarrollar funciones en Python que reciben como parámetro de entrada una petición (request) y devuelven una plantilla u otra respuesta HTTP.
  - Template: Conocimientos de HTML, CSS y JavaScript para la creación de interfaces de usuario, así como del lenguaje de plantillas de Django para conectar vistas con templates y heredar estructuras.

2. Frontend y visualización de datos

- Librerías de visualización: Conocimiento de Plotly para la generación de gráficas interactivas, estáticas y dinámicas, incluyendo gráficos de puntos, lineales, mapas de calor y representaciones tridimensionales.
- Tratamiento de datos: Dominio de librerías como NumPy, Pandas, Seaborn y SciPy para el procesamiento y análisis de datos de los modelos neuronales.

3. Simulación y ejecución de modelos

- Machine learning: Conocimientos básicos en machine learning para comprender y trabajar con los modelos implementados en la plataforma.
- Automatización y sistema: Familiaridad con librerías como subprocess, os y utils para la ejecución de procesos y manejo del sistema.
- Contenedores: Nociones de Docker para la gestión de entornos de ejecución y la reproducibilidad de los modelos.

4. Sistemas inteligentes (CI15)

- Conocimiento y aplicación de los principios fundamentales y técnicas básicas de los sistemas inteligentes y su aplicación práctica, incluyendo la comprensión de algoritmos de aprendizaje automático, razonamiento basado en conocimiento y sistemas adaptativos.
- Capacidad para seleccionar y aplicar la técnica de inteligencia artificial más adecuada según el problema a resolver, evaluando sus ventajas y limitaciones en el contexto del proyecto.

- Integración de componentes de inteligencia artificial en aplicaciones software, comprendiendo el ciclo de vida completo desde el diseño hasta el despliegue de modelos.

5. Diseño de interfaces y experiencia de usuario (CI17)

- Capacidad para diseñar y evaluar interfaces persona-computador que garanticen la accesibilidad y usabilidad a los sistemas, servicios y aplicaciones informáticas, aplicando estándares y directrices de diseño centrado en el usuario.
- Conocimiento de técnicas de evaluación de usabilidad, incluyendo pruebas con usuarios, análisis heurístico y métricas de experiencia de usuario para la mejora iterativa de las interfaces.
- Aplicación de principios de diseño responsive y adaptativo para garantizar una experiencia consistente en diferentes dispositivos y tamaños de pantalla.
- Implementación de pautas de accesibilidad (WCAG) para asegurar que la aplicación sea utilizable por personas con diversas capacidades y discapacidades.

1.5. Alineamiento con los objetivos de desarrollo sostenible

El presente proyecto se alinea con el ODS 4: Educación de Calidad, contribuyendo a cuatro metas específicas que abordan tanto el acceso como la calidad de la educación científica.

En primer lugar, en relación con la Meta 4.3 (Acceso a educación superior), la plataforma facilita el acceso a tecnologías avanzadas de visualización a estudiantes de posgrado e investigadores, reduciendo la barrera de entrada a la neurociencia computacional. Tradicionalmente, el análisis de modelos computacionales en este campo requiere conocimientos profundos de programación y manejo de datos, lo que limita el acceso a investigadores noveles o de otras disciplinas. Nuestra herramienta elimina esta barrera al ofrecer una interfaz gráfica e interactiva que permite visualizar resultados complejos sin necesidad de escribir código, democratizando así el acceso al conocimiento especializado y promoviendo la igualdad de oportunidades en el ámbito científico.

En segundo lugar, respecto a la Meta 4.4 (Competencias técnicas), los usuarios mejoran sus habilidades en programación, análisis de datos y modelado computacional al interactuar con una herramienta que transforma datos complejos en representaciones gráficas intuitivas. Al utilizar la plataforma, los investigadores no solo obtienen visualizaciones de sus modelos, sino que también desarrollan competencias clave en el manejo de herramientas digitales, interpretación de gráficas científicas y comprensión de la relación entre parámetros de entrada y resultados de salida. Estas habilidades son transferibles a otros ámbitos de la investigación y mejoran su empleabilidad en el sector científico-tecnológico, contribuyendo directamente a la formación de capital humano avanzado.

En tercer lugar, la plataforma da respuesta a la Meta 4.7 (Educación multidisciplinar y colaborativa), al permitir la comparación de modelos entre distintos usuarios y contar con una interfaz accesible que fomenta el aprendizaje colaborativo y el intercambio de conocimientos entre perfiles de diversas disciplinas (Biología, Psicología, Ingeniería, Matemáticas). La neurociencia computacional es inherentemente multidisciplinar, y esta herramienta actúa como un puente que conecta a investigadores de distintas áreas, facilitando el diálogo científico y la transferencia de conocimiento. Además, al promover una educación abierta y colaborativa, la plataforma contribuye a formar científicos con una visión integral y sostenible de la investigación, capaces de abordar problemas complejos desde múltiples perspectivas.

Finalmente, en cuanto a la Meta 4.a (Entornos de aprendizaje inclusivos y eficaces), la plataforma web constituye un entorno digital seguro, accesible e intuitivo, optimizado para distintos dispositivos, que garantiza una experiencia de aprendizaje eficaz para todos los usuarios, independientemente de su nivel de conocimiento previo. El diseño de la interfaz ha priorizado la usabilidad y la accesibilidad, con especial atención a la claridad visual, la navegación sencilla y la adaptabilidad a diferentes tamaños de pantalla (ordenador, tableta, móvil). De este modo, se asegura que investigadores con distintos niveles de experiencia tecnológica, así como aquellos con limitaciones de recursos o de conectividad, puedan beneficiarse igualitariamente de las funcionalidades de la plataforma, promoviendo así una ciencia más inclusiva y equitativa.

2. Planificación y Metodología

Para el desarrollo de esta plataforma se adoptó la metodología SCRUM debido a su flexibilidad y capacidad de adaptación ante cambios e imprevistos surgidos durante el proyecto. Aunque la planificación inicial contemplaba una ejecución continua de aproximadamente cuatro meses, el desarrollo se llevó a cabo a lo largo de dicho periodo con diversas interrupciones y reajustes derivados de circunstancias académicas y técnicas. En este contexto, SCRUM permitió reorganizar las tareas y prioridades sin comprometer los avances ya alcanzados, facilitando una evolución progresiva de la plataforma. La gestión del proyecto se estructuró mediante una pila de producto compuesta por épicas, historias de usuario, tareas técnicas y actividades de investigación destinadas a analizar la viabilidad e integración de distintos modelos computacionales de difusión del óxido nítrico (NO). Asimismo, se emplearon los mecanismos propios de la metodología, como la planificación de sprint, el seguimiento del trabajo realizado y las revisiones periódicas, con el objetivo de garantizar la correcta evolución del sistema. El desarrollo se organizó inicialmente en sprints de dos semanas de duración. No obstante, debido a las interrupciones producidas durante el proyecto, la planificación fue adaptándose en función de la disponibilidad y de los objetivos prioritarios de cada etapa. A lo largo de los cuatro meses de trabajo se realizaron reuniones de seguimiento y revisiones periódicas que permitieron evaluar el estado de la plataforma, detectar incidencias y definir las acciones necesarias para continuar con el desarrollo de manera eficiente.

2.1. Roles del desarrollador del proyecto

El desarrollador ha tenido los siguientes roles: Dado que el proyecto ha sido desarrollado por una sola persona, el desarrollador ha asumido múltiples roles para garantizar el éxito del proyecto. A continuación, se detallan los roles desempeñados:

Cuadro 1 Roles desempeñados en el proyecto

Rol	Responsabilidades
Desarrollador	▪ Implementación del código fuente ▪ Configuración del entorno de desarrollo ▪ Integración de librerías y dependencias ▪ Diseño e implementación de la interfaz de usuario ▪ Desarrollo del backend con Django ▪ Configuración de Docker y contenedores ▪ Creación de gráficas y visualizaciones ▪ Implementación del simulador ▪ Gestión de la base de datos PostgreSQL ▪ Realización de pruebas funcionales ▪ Depuración y corrección de errores

Continúa en la siguiente página

Tabla continua: Roles desempeñados	
Rol	**Responsabilidades**
Product Owner	▪ Definición de los requisitos del proyecto ▪ Priorización del backlog del producto ▪ Gestión de la pila de sprints ▪ Toma de decisiones sobre funcionalidades ▪ Validación de los entregables ▪ Ajuste de prioridades según necesidades ▪ Definición de criterios de aceptación ▪ Gestión de expectativas y alcance
Scrum Master	▪ Planificación de sprints ▪ Seguimiento del progreso del proyecto ▪ Gestión de impedimentos ▪ Aseguramiento de la metodología SCRUM ▪ Facilitación de reuniones ▪ Control de tiempos y entregas
Analista	▪ Investigación del estado del arte ▪ Análisis de requisitos funcionales ▪ Estudio de modelos de difusión del NO ▪ Definición de casos de uso ▪ Documentación del sistema

2.2. Herramientas usadas para la planificación

Debido a problemas con JIRA no ha sido posible el uso de esta herramienta por lo que se ha optado por el uso del word para la realización del product backlog,sprints backlog y sprints reviews.

2.3. Planificación Temporal con Diagrama de Gantt

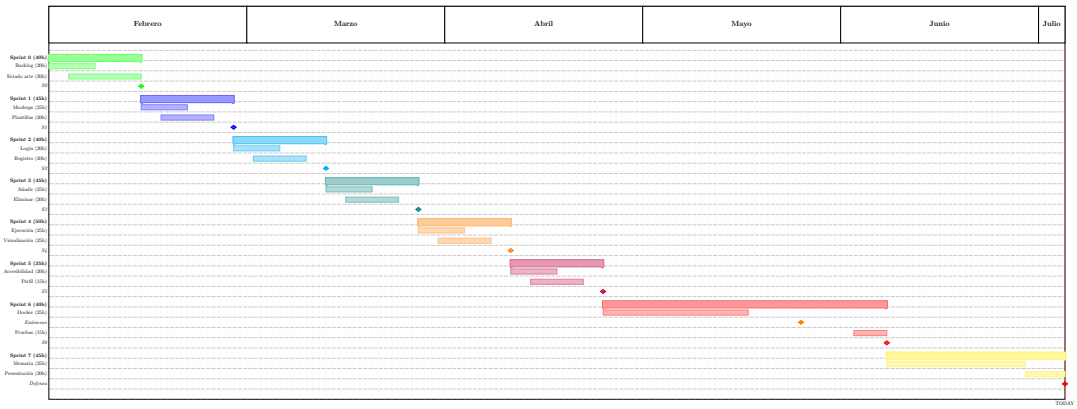


Figura 2 Planificación de sprints del proyecto (Febrero - Julio 2026)

2.4. Resumen del Product Backlog

El desarrollo se organizó en 8 sprints de dos semanas de duración cada uno, siguiendo la metodología SCRUM. En la Tabla 2 se resumen los objetivos, las historias de usuario y las principales tareas técnicas de cada sprint.

Cuadro 2 Resumen del Product Backlog por sprints

Sprint	Objetivo y funcionalidades	Tareas técnicas destacadas
0: Preparación	▪ Diseño de mockups de la plataforma ▪ Elaboración del Product Backlog ▪ Estado del arte y aprendizaje del framework Django	▪ Definición de historias de usuario iniciales ▪ Configuración del entorno de desarrollo
1: Maquetación y BD	▪ Desarrollo de plantillas HTML/CSS ▪ Creación de la base de datos PostgreSQL ▪ Aprendizaje de Docker	▪ Modelado de la base de datos ▪ Configuración de Docker Compose ▪ Priorización de historias para el siguiente sprint

Continúa en la siguiente página

Tabla 2 – Continuación

Sprint	Objetivo y funcionalidades	Tareas técnicas destacadas
2: Gestión de usuarios	▪ Inicio y cierre de sesión ▪ Registro de usuarios ▪ Recuperación de contraseña	▪ Implementación del modelo de usuarios ▪ Manejo de errores en autenticación ▪ Validación de formularios
3: Gestión de modelos	▪ Añadir modelos de forma permanente ▪ Eliminar modelos de forma permanente	▪ Creación de la tabla de modelos ▪ Validación de archivos subidos ▪ Manejo de errores en operaciones CRUD
4: Simulador + Visualización	▪ Subida y ejecución de modelos ▪ Extracción de datos de entrada y salida ▪ Visualización de gráficas (estáticas y dinámicas) ▪ Exportación de gráficas	▪ Integración con `subprocess` ▪ Análisis AST para extracción de variables ▪ Implementación de gráficas con Plotly ▪ Manejo de errores en simulación
5: Perfil y accesibilidad	▪ Cambio de contraseña ▪ Eliminación de cuenta ▪ Edición de perfil ▪ Modo claro/oscuro ▪ Aumento de tamaño de letra	▪ Implementación de preferencias de usuario ▪ Atributos WAI-ARIA ▪ Manejo de errores en ajustes
6: Pruebas y despliegue	▪ Implementación de middleware ▪ Pruebas de integración con diferentes modelos ▪ Refactorización de código ▪ Dockerización y despliegue en servidor	▪ Configuración final de Docker ▪ Pruebas funcionales con Django Test ▪ Validación de accesibilidad con WAVE

Continúa en la siguiente página

Tabla 2 – Continuación

Sprint	Objetivo y funcionalidades	Tareas técnicas destacadas
7: Defensa	▪ Elaboración de la memoria final ▪ Preparación de la presentación ▪ Difusión científica de los resultados	▪ Redacción de conclusiones ▪ Generación de material gráfico para la defensa

2.5. Gráficas de Seguimiento del Proyecto

2.5.1. Gráfica Burndown

La gráfica Burndown muestra la evolución del trabajo pendiente a lo largo de los sprints, comparando el progreso real con el ritmo ideal esperado.

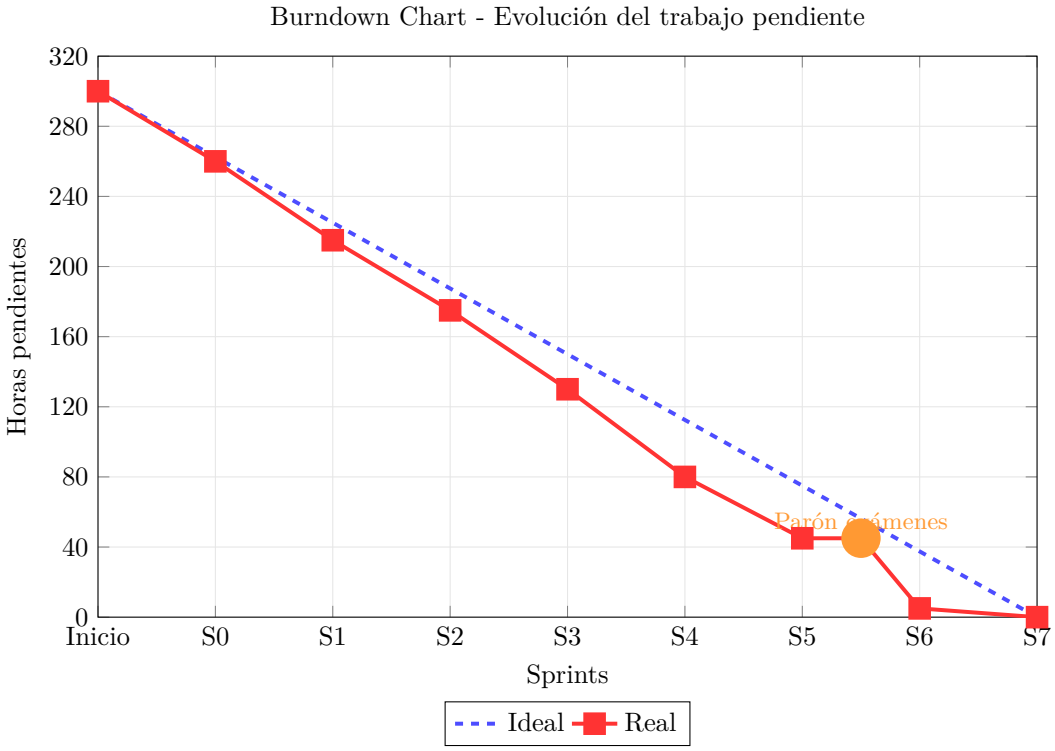


Figura 3 Burndown Chart - Seguimiento del trabajo pendiente

2.5.2. Gráfica Burnup

La gráfica Burnup muestra el trabajo completado acumulado a lo largo del proyecto, permitiendo visualizar el progreso real frente al objetivo planificado.

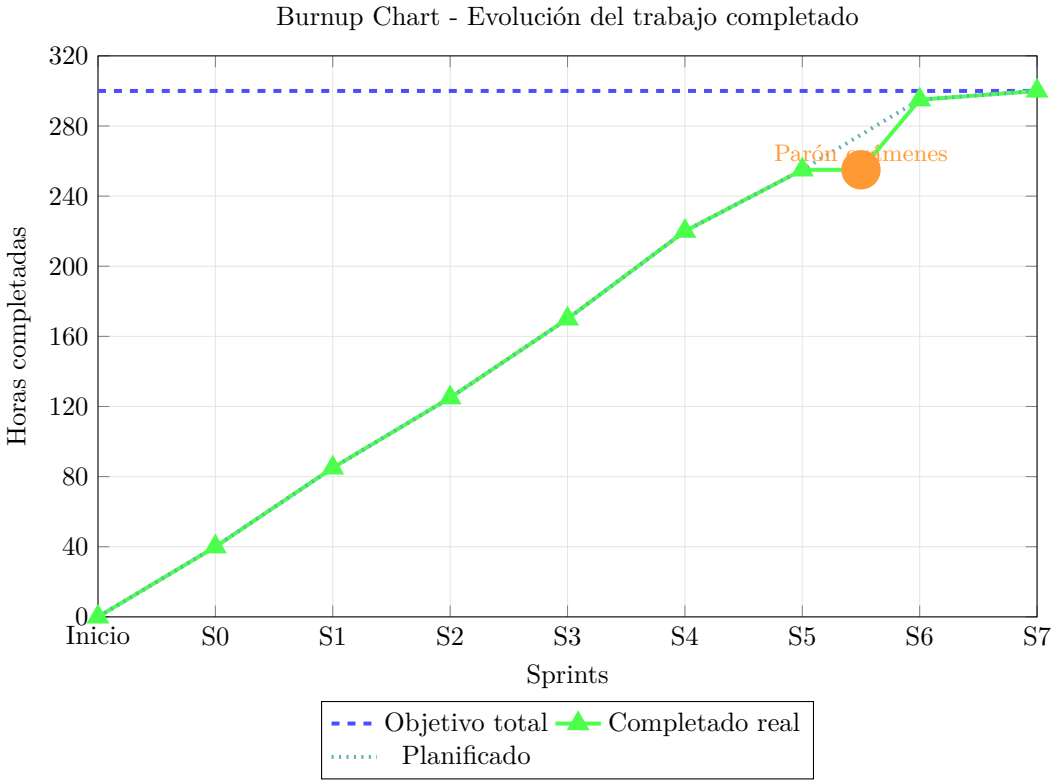


Figura 4 Burnup Chart - Seguimiento del trabajo completado

3. Desarrollo

En este capítulo se explica de forma detallada el entorno de desarrollo, dependencias, estructura del proyecto y decisiones clave que se han planteado a lo largo de todo el proceso.

3.1. Tecnologías empleadas

Para el desarrollo de la plataforma se ha utilizado un conjunto de tecnologías orientadas al desarrollo web, almacenamiento de datos, despliegue mediante contenedores y visualización de resultados científicos. La Tabla 3 describe las principales tecnologías empleadas.

Cuadro 3 Tecnologías empleadas en el desarrollo

Tecnología	**Descripción**
Python	Python es un lenguaje de programación de tipo interpretado, funcional, orientado a objetos e interactivo.
	Continúa en la siguiente página

Tabla continua: Tecnologías empleadas	
Tecnología	**Descripción**
Django	Django es un framework de desarrollo web para Python que permite implementar tanto la lógica de negocio del backend como la gestión de las interfaces web. Su elección se debe a que los modelos de simulación utilizados en el proyecto están desarrollados en Python, lo que facilita su integración con la aplicación web y reduce la complejidad del desarrollo. Además, Django incorpora mecanismos de autenticación, gestión de usuarios, administración de bases de datos y protección frente a vulnerabilidades comunes, permitiendo desarrollar aplicaciones robustas y seguras.
PostgreSQL	PostgreSQL es el sistema gestor de bases de datos relacional seleccionado para el proyecto. Su elección se debe a su fiabilidad, rendimiento y compatibilidad con Django mediante el uso de su sistema ORM (Object Relational Mapping). La base de datos almacena la información relacionada con los usuarios, los modelos cargados en la plataforma, los datos de perfil y los resultados asociados a las simulaciones realizadas.
Subprocess	El módulo subprocess permite lanzar nuevos procesos, conectarse a sus pipes de entrada/salida/error y obtener sus códigos de resultado. Este módulo está destinado a reemplazar múltiples módulos y funciones previamente existentes.
AST	El módulo ast ayuda a las aplicaciones de Python a procesar árboles de la gramática de sintaxis abstracta de Python. La sintaxis abstracta en sí misma puede cambiar con cada versión de Python; este módulo ayuda a descubrir mediante programación cómo se ve la gramática actual.
Plotly.js	Plotly.js es una biblioteca JavaScript especializada en la creación de visualizaciones científicas e interactivas. Se utiliza para representar gráficamente los resultados obtenidos durante la ejecución de los modelos de difusión. Entre los tipos de visualización soportados se encuentran gráficos de líneas, diagramas de dispersión, mapas de calor y representaciones tridimensionales, permitiendo analizar los resultados de forma visual e intuitiva.
Continúa en la siguiente página	

Tabla continua: Tecnologías empleadas	
Tecnología	**Descripción**
Docker	Docker es una plataforma de contenedorización utilizada para facilitar el despliegue de la aplicación en el servidor del grupo de investigación de Neurociencia Computacional de la Universidad de Las Palmas de Gran Canaria. Su utilización permite encapsular todos los componentes necesarios para la ejecución del sistema, incluyendo la aplicación web, la base de datos y las herramientas de administración. Esto garantiza que el comportamiento de la aplicación sea consistente independientemente del entorno donde se despliegue.
Git	Git es un sistema de control de versiones que realiza un seguimiento de los cambios en los archivos.
GitHub	GitHub es una plataforma basada en la nube donde puedes almacenar, compartir y trabajar junto con otros usuarios para escribir código. Almacenar tu código en un "repositorio" en GitHub te permite: ▪ Presentar o compartir el trabajo. ▪ Seguir y administrar los cambios en el código a lo largo del tiempo. ▪ Dejar que otros usuarios revisen el código y realicen sugerencias para mejorarlo. ▪ Colaborar en un proyecto compartido, sin preocuparse de que los cambios afectarán al trabajo de los colaboradores antes de que esté listo para integrarlos. El trabajo colaborativo, una de las características fundamentales de GitHub, es posible gracias al software de código abierto Git, en el que se basa GitHub.

3.2. Dependencias del proyecto

Las principales dependencias utilizadas durante el desarrollo se muestran en la Tabla 4:

Cuadro 4 Dependencias del proyecto

Dependencia	**Descripción**
Python 3.12	Lenguaje principal de desarrollo
Django	Framework web
PostgreSQL	Sistema gestor de bases de datos
Plotly	Biblioteca de visualización de datos
python-dotenv	Gestión de variables de entorno
Docker	Contenedorización y despliegue
Subprocess	Ejecución de procesos externos

Continúa en la siguiente página

Tabla continua: Dependencias del proyecto	
Dependencia	**Descripción**
AST	Análisis de código Python

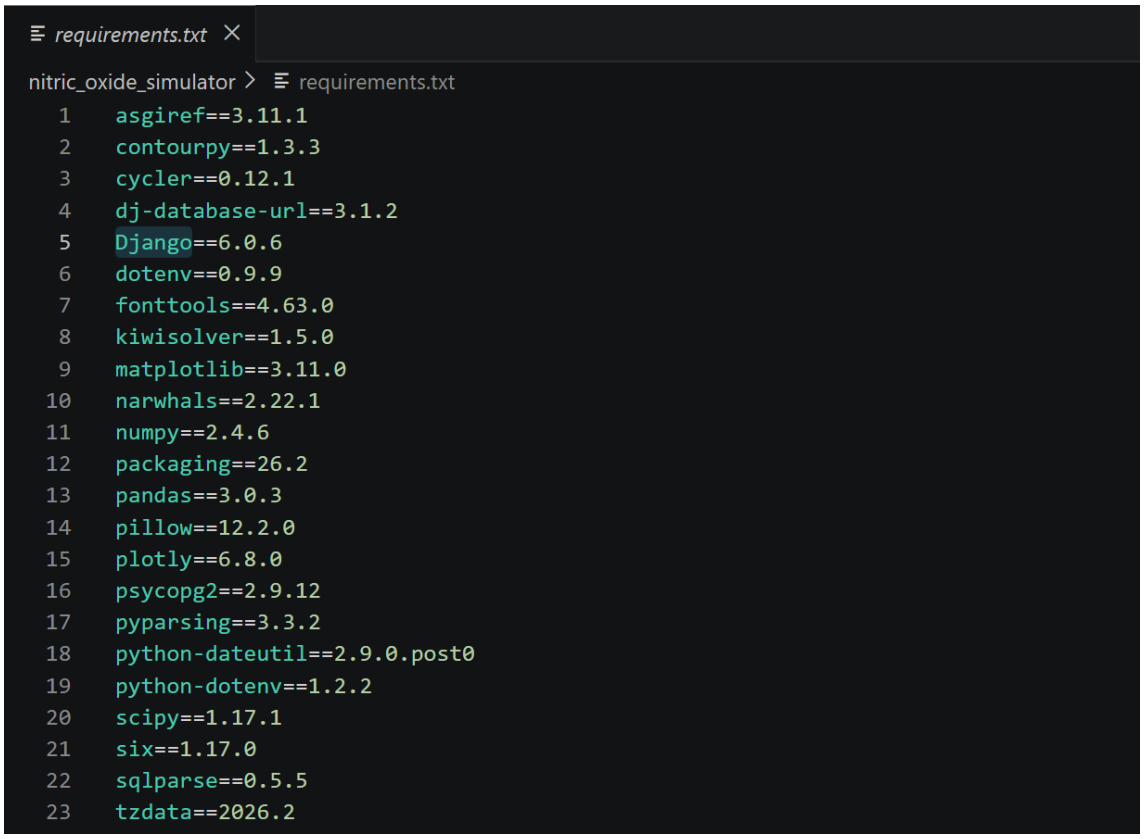


Figura 5 Archivo de dependencias del proyecto

3.3. Entorno de desarrollo

El desarrollo del proyecto se ha realizado utilizando Visual Studio Code como entorno de desarrollo integrado. Esta herramienta proporciona soporte para Python, Docker y desarrollo web, facilitando la implementación y mantenimiento del sistema. Para el control de versiones se ha utilizado Git, permitiendo gestionar de forma eficiente la evolución del código fuente durante el desarrollo del proyecto.

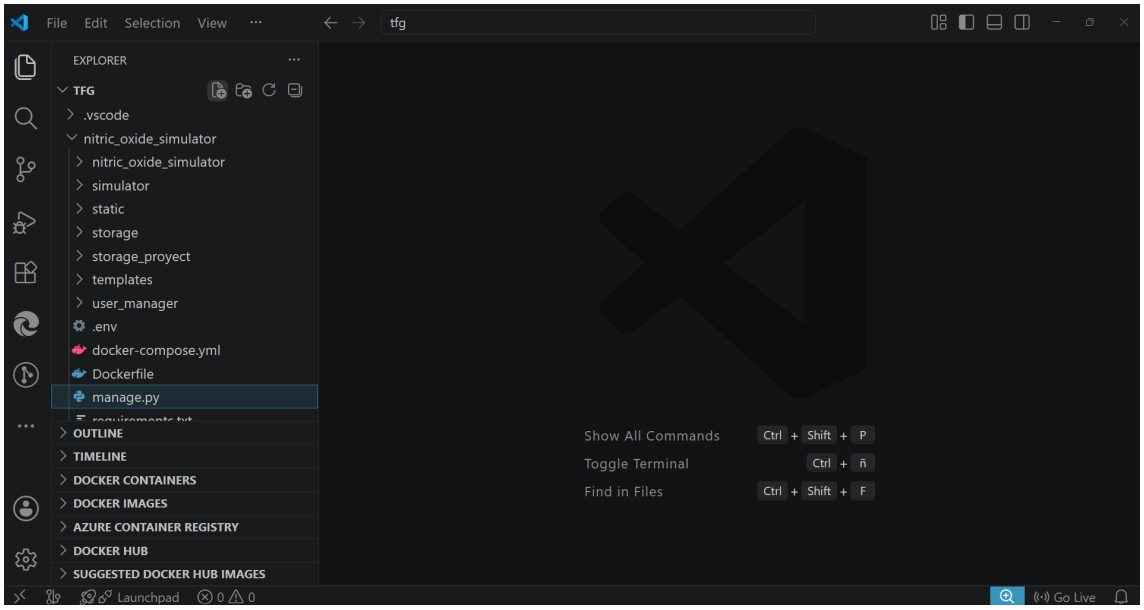


Figura 6 Entorno de trabajo en Visual Studio Code

3.4. Entorno de pruebas y validación

Con el objetivo de garantizar el correcto funcionamiento de la aplicación, se realizaron diferentes pruebas utilizando el sistema de testing proporcionado por Django.

Las pruebas implementadas cubren las siguientes funcionalidades: registro de usuarios, inicio de sesión, gestión de perfiles, almacenamiento de modelos, eliminación de modelos, ejecución de simulaciones y generación de gráficas.

Asimismo, se realizaron pruebas de compatibilidad en distintos navegadores web, incluyendo Google Chrome, Mozilla Firefox y Vivaldi. Para la validación de la accesibilidad se empleó la herramienta WAVE Evaluation Tool, verificando aspectos relacionados con la estructura semántica de la interfaz, la accesibilidad de los componentes y el contraste de colores.

3.5. Estructura del proyecto

Al usar el framework Django, se emplea una estructura de directorios vertical, haciendo que el proyecto tenga la posibilidad de crecer a futuro creando por cada directorio una aplicación independiente que luego se pueda integrar al proyecto final.

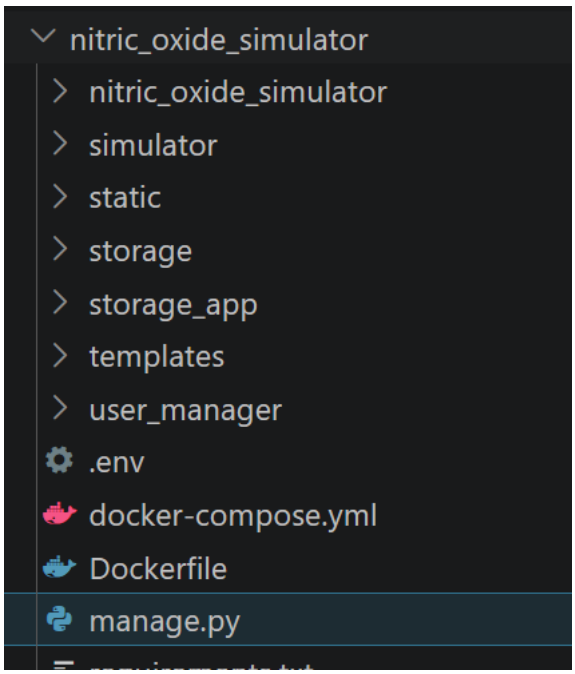


Figura 7 Estructura de directorios del proyecto

4. Diseño

En este capítulo se explican las decisiones de diseño a nivel UX/UI y el diseño lógico de la plataforma web de simulación y visualización de los modelos de difusión del NO. Se describen de forma más detallada las partes gráficas de la plataforma, los patrones de diseño y su justificación, y la visualización de los bocetos y mockups finales que llevaron a la parte visual actual. Además, se explican los aspectos integrados de accesibilidad dentro del proyecto.

4.1. Características de la interfaz de usuario (UI)

La interfaz gráfica de usuario (UI) del proyecto ha sido diseñada con el objetivo de proporcionar una experiencia de uso intuitiva, accesible y eficiente para estudiantes e investigadores del ámbito de la neurociencia computacional. Durante el desarrollo se ha priorizado la simplicidad visual, la facilidad de navegación y la correcta visualización de los resultados obtenidos a partir de las simulaciones realizadas.

Una de las funcionalidades incorporadas en la aplicación es la posibilidad de alternar entre modo claro y modo oscuro. Esta característica permite adaptar la interfaz a las preferencias visuales de cada usuario y a diferentes condiciones de iluminación, contribuyendo a reducir la fatiga visual durante sesiones prolongadas de trabajo. Ambas paletas de colores han sido evaluadas mediante herramientas de accesibilidad como WAVE Evaluation Tool, verificando el cumplimiento de los criterios de contraste establecidos por las WCAG (Web Content Accessibility Guidelines).

La paleta de colores claros, referenciada en la 8, está compuesta por una serie de colores seleccionados para transmitir profesionalidad, claridad y comodidad visual. Como colores primarios dominantes se emplean los tonos #1e4766 (Cello) y #2a4045 (Pickled Bluewood), utilizados principalmente en barras de navegación, botones principales y títulos relevantes. Para los fondos principales se utilizan los colores #f6f7fb y #ecf0f8 (White Lilac), que proporcionan una apariencia limpia y reducen la fatiga visual en comparación con un blanco puro. El texto principal se representa mediante los colores #000000 y #040404, garantizando una adecuada legibilidad y contraste sobre fondos claros.

Como colores secundarios se emplean los tonos #72abd5 (Danube), #8eb5c6 y #93b8c6 (Nepal), destinados a resaltar enlaces, iconos decorativos y elementos informativos. Asimismo, se utilizan colores neutros de apoyo como #628684 (Juniper), #838484 y #7c8c8c (Gunsmoke), adecuados para bordes, subtítulos, información secundaria y elementos de separación visual. Para indicar acciones de confirmación se utiliza el color Juniper, mientras que las advertencias y errores se representan mediante los colores #e6a23c y #c0392b, respectivamente.

La combinación de azules, grises y tonos verde-azulados transmite sensaciones de confianza, estabilidad, profesionalidad y tecnología, características especialmente adecuadas para aplicaciones científicas y plataformas de análisis de datos.

Por otra parte, la paleta de colores oscuros, referenciada en la 9, ha sido diseñada para ofrecer una experiencia visual cómoda en entornos con poca iluminación. Los colores primarios dominantes y de acción corresponden a los tonos #1d88e4, #388cd1 y #258cdf (Curious Blue), utilizados en botones principales, enlaces activos y elementos seleccionados. Los fondos principales utilizan los colores #1b242c (Mirage) y #23313a (Ebony Clay), que proporcionan una apariencia oscura sin recurrir al negro absoluto, reduciendo así la fatiga visual. El texto principal se representa mediante el color #ffffff (White), garantizando un elevado nivel de contraste respecto al fondo.

Los colores secundarios de esta paleta incluyen #045383 (Venice Blue) y #115884 (Chathams Blue), empleados en tarjetas, contenedores secundarios y estados interactivos de los componentes. Asimismo, se utilizan tonos neutros como #a1bcc8 (Casper), #c3d4dc (Jungle Mist) y #708089 (Slate Gray) para bordes, iconografía complementaria y textos secundarios. Para las acciones de confirmación se emplean los colores #7aa2b7 (Neptune) y #51758b (Smalt Blue), mientras que las advertencias y errores utilizan tonalidades ámbar y rojo coral para destacar adecuadamente sobre fondos oscuros sin romper la coherencia visual de la interfaz.

La combinación de azul eléctrico sobre fondos oscuros transmite una estética tecnológica, moderna y avanzada, ampliamente utilizada en aplicaciones de análisis de datos, paneles de control e interfaces profesionales.

En cuanto a la tipografía, se ha seleccionado la fuente Arial debido a su elevada legibilidad y amplia compatibilidad entre plataformas. El tamaño de fuente utilizado es de 11 puntos para el contenido general, adaptándose el color del texto a la paleta activa. En el modo claro se emplean tonos oscuros sobre fondos claros, mientras que en el modo oscuro se utilizan tonos claros sobre fondos oscuros, garantizando siempre niveles adecuados de contraste y legibilidad.

Con el objetivo de mejorar la accesibilidad, la aplicación incorpora atributos WAI-ARIA en los componentes interactivos más relevantes. Esto permite que los lectores de pantalla interpreten correctamente los elementos de la interfaz y facilita el acceso a la aplicación por parte de personas con discapacidad visual. Adicionalmente, el uso de etiquetas HTML semánticas contribuye a una mejor comprensión de la estructura de la página por parte de las tecnologías de asistencia.

La iconografía utilizada desempeña un papel importante en la interacción con el usuario. Cada funcionalidad principal se encuentra acompañada de un icono representativo que facilita su identificación visual. La Tabla 5 muestra los iconos utilizados y su significado.

Cuadro 5 Iconos de la aplicación y su funcionalidad

Icono	Nombre	Funcionalidad
🗑	Eliminar modelo	Permite eliminar un modelo almacenado de forma permanente.
☁	Subir modelo	Permite cargar un nuevo modelo desde el dispositivo del usuario.
🗎	Añadir modelo	Añade un modelo a la biblioteca personal del usuario.
👤	Perfil	Accede a la sección de gestión del perfil de usuario.
▶	Simular modelo	Ejecuta la simulación del modelo seleccionado.
🖫	Exportar gráficas	Permite descargar las gráficas generadas en formato JPG.
✎	Editar perfil	Modifica los datos personales del usuario.

La estructura general de la interfaz se ha implementado siguiendo los principios del HTML semántico, favoreciendo tanto la organización interna del código como la accesibilidad. En la parte superior se encuentra el encabezado o header, cuyo color se adapta automáticamente a la paleta seleccionada. Este encabezado incluye las opciones de inicio de sesión, registro de usuarios y cambio entre modo claro y oscuro, además de diversos elementos gráficos relacionados con la neurociencia computacional. Una vez autenticado, el usuario dispone de accesos directos a las secciones de almacenamiento, perfil personal y configuración visual.

La zona central de la aplicación corresponde al elemento `main`, donde se presenta el contenido principal. Dependiendo de la sección seleccionada, esta área muestra formularios de autenticación, herramientas para la carga de modelos, tablas de almacenamiento, opciones de gestión del perfil o el panel principal de simulación. Este último constituye el núcleo funcional de la plataforma, permitiendo configurar los parámetros de entrada de los modelos y visualizar los resultados obtenidos.

Los formularios incorporados en la aplicación incluyen mecanismos de validación que permiten detectar errores antes del envío de los datos al servidor. Cuando se produce una incidencia, la interfaz muestra mensajes descriptivos que informan al usuario sobre el problema detectado y las acciones necesarias para corregirlo. Del mismo modo, las operaciones realizadas generan mensajes de confirmación o advertencia que proporcionan retroalimentación inmediata y mejoran la experiencia de uso.

Uno de los componentes más importantes de la plataforma es el dashboard de simulación. Este panel permite representar gráficamente los resultados obtenidos tras la ejecución de los modelos de difusión. Las gráficas muestran la evolución temporal de la concentración calculada por el modelo a lo largo del intervalo de simulación definido por el usuario. Esta representación visual facilita

la interpretación de los resultados y permite analizar el comportamiento dinámico del fenómeno estudiado de forma rápida e intuitiva.

En la parte inferior de todas las páginas se encuentra el pie de página o footer, que mantiene una apariencia uniforme en toda la aplicación mediante el uso de un color azul corporativo. Este elemento proporciona consistencia visual y contribuye a mantener una estructura homogénea en todas las secciones de la plataforma.

Durante el diseño de la interfaz también se han aplicado principios de usabilidad orientados a reducir la curva de aprendizaje de los usuarios. Para ello, se ha mantenido una distribución consistente de los elementos de navegación, una organización clara de las funcionalidades y una iconografía fácilmente reconocible. Asimismo, se ha buscado minimizar el número de acciones necesarias para ejecutar simulaciones, consultar resultados o gestionar modelos, mejorando la eficiencia general de la plataforma.

Gracias a esta estructura, la aplicación proporciona un entorno de trabajo intuitivo, accesible y funcional que permite configurar, ejecutar y analizar modelos de difusión dentro de un único sistema integrado. La combinación de criterios de accesibilidad, usabilidad y diseño visual facilita que usuarios con diferentes niveles de experiencia puedan utilizar la plataforma de manera eficiente y comprender los resultados obtenidos durante las simulaciones.

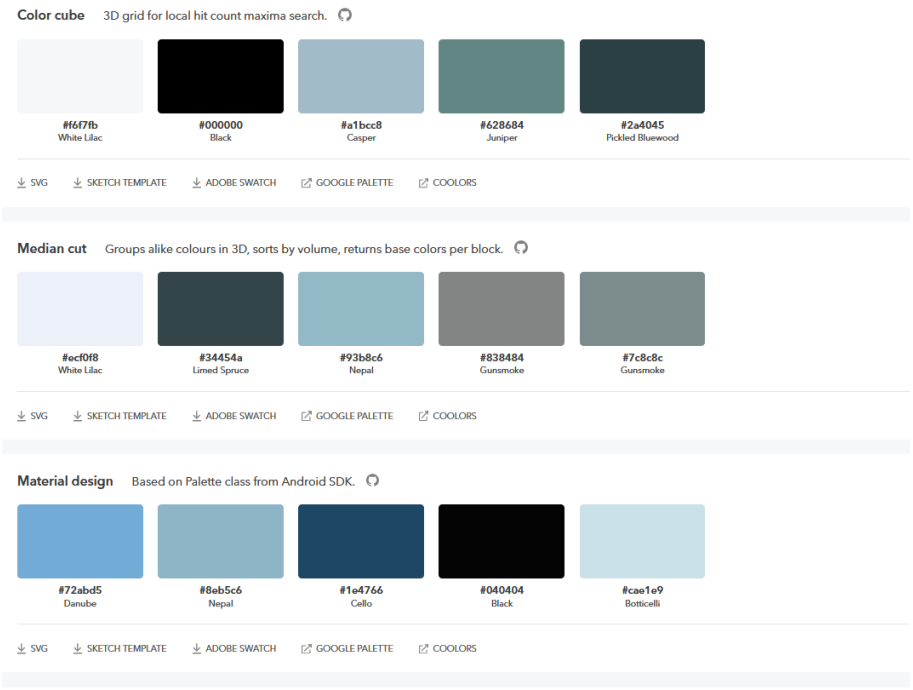


Figura 8 Paleta de colores claro

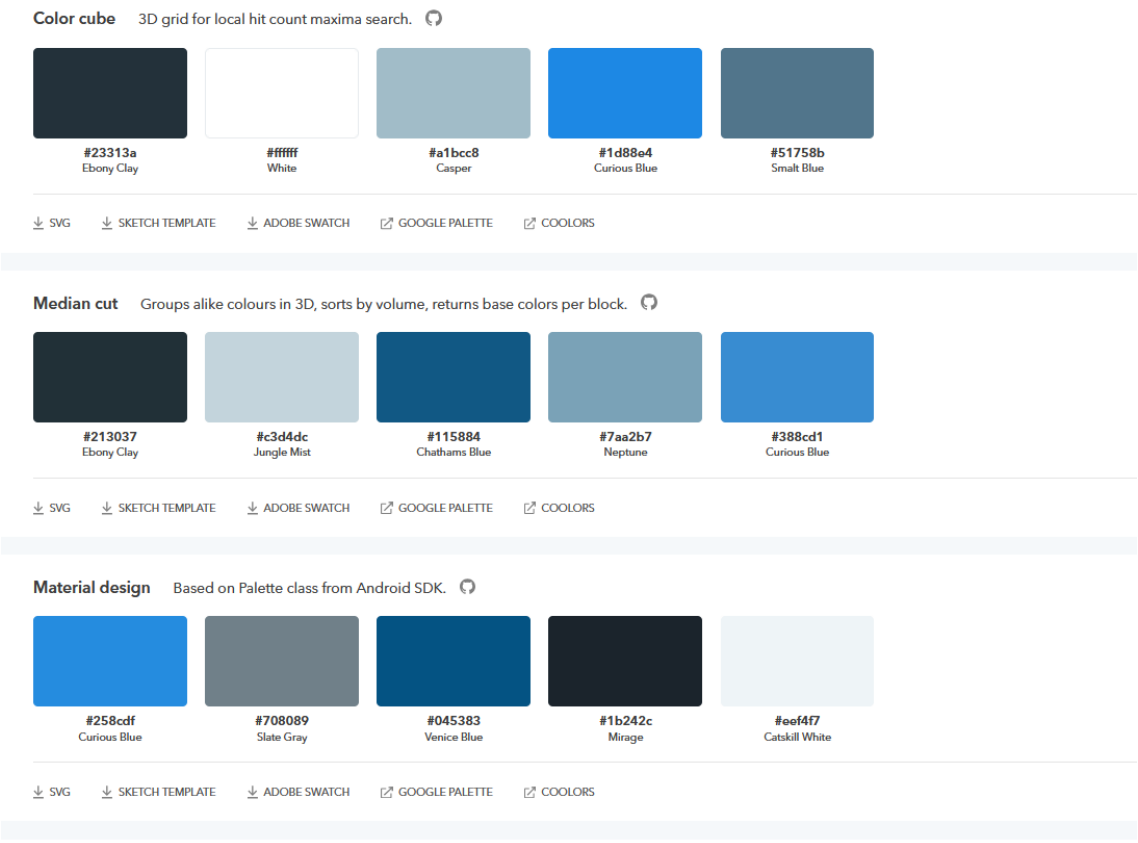


Figura 9 Paleta de colores oscuro

4.2. Patrones de diseño y decisiones

4.2.1. Patrones de diseño

La idea de usar patrones de diseño es esencial para mejorar la esencia del software elaborado. Por esta razón, se ha seleccionado para este proyecto el siguiente conjunto de patrones de diseño, descritos en la Tabla 6:

Cuadro 6 Patrones de diseño utilizados

Patrón de diseño	**Descripción**
Simple Factory	El patrón se basa en una función que crea objetos como si fuera una factoría.
Decorator	El patrón se basa en añadir funcionalidades extra a objetos de forma encapsulada.
Iterator	El patrón se basa en iterar en una estructura de datos como pilas, colas, matrices, vectores, etc.
Builder	El patrón se basa en la creación de varios productos a partir de una base.

4.2.2. Justificación de los patrones de diseño

La justificación de cada patrón de diseño se explica en la Tabla 7:

Cuadro 7 Justificación de los patrones de diseño

Patrón de diseño	**Explicación**
Simple Factory	Centraliza la creación de gráficas similares en un solo lugar, evitando que el código cliente tenga que saber cómo se crean.
Decorator	Añade funcionalidades extra a funciones sin modificar su código original, siguiendo el principio Open/Closed.
Iterator	Recorre colecciones de datos que son los resultados de los modelos, como matrices o vectores de intervalo de tiempo y cantidades de concentración, sin exponer su estructura interna, proporcionando una interfaz uniforme.
Builder	Construye objetos como las gráficas paso a paso, separando la construcción de la representación final.

4.3. Prototipos y Mockups

En primer lugar, se muestran los primeros bocetos elaborados pensando en cómo se estructuraría visualmente la plataforma web.

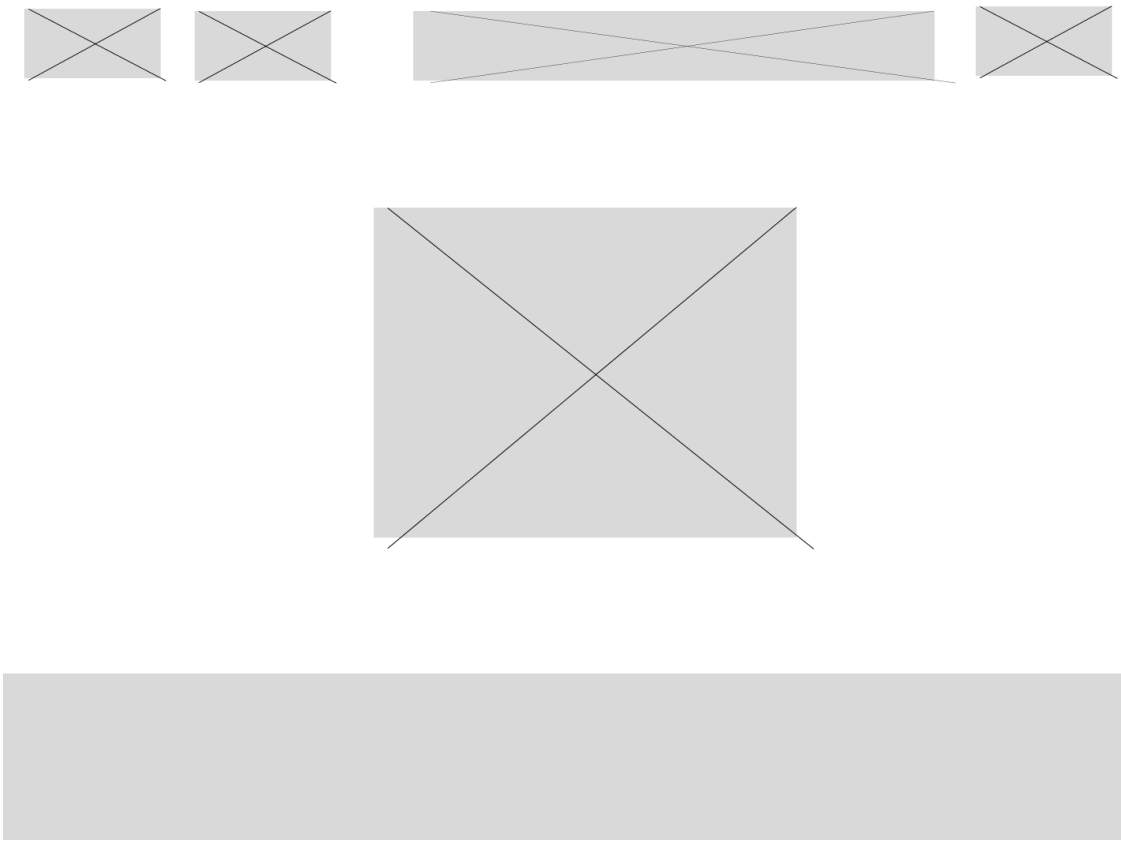


Figura 10 Página de inicio (boceto inicial)

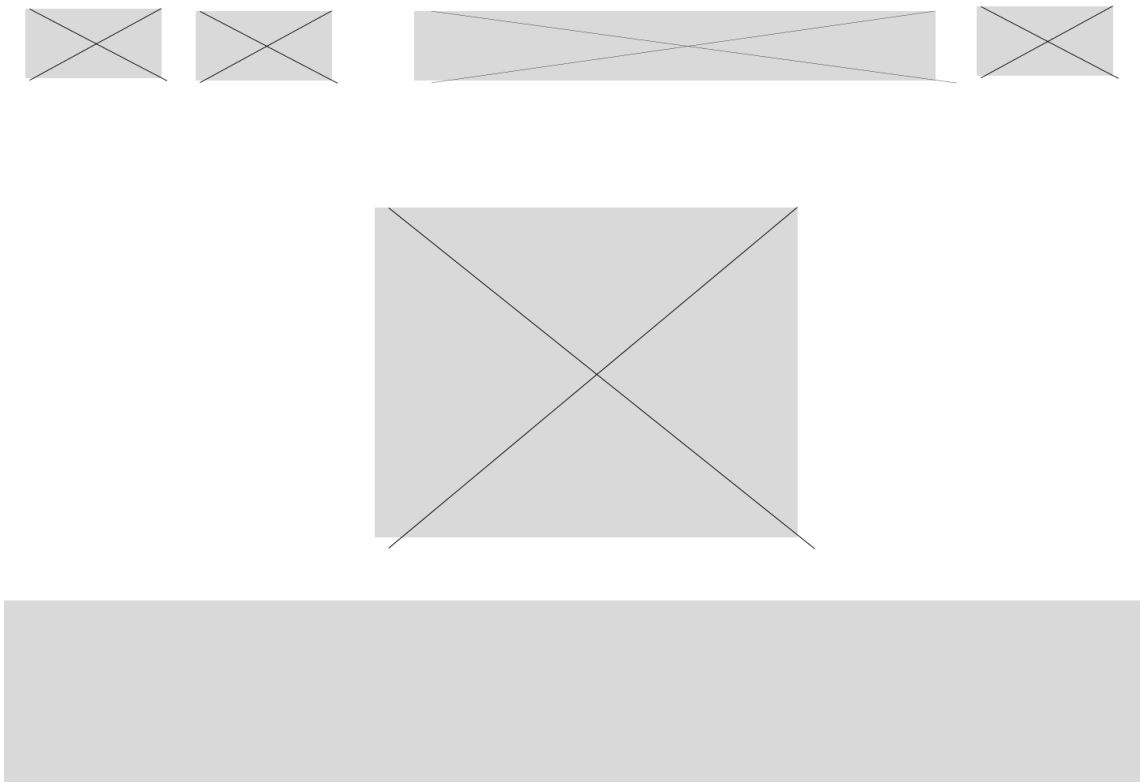


Figura 11 Iniciar sesión (boceto inicial)

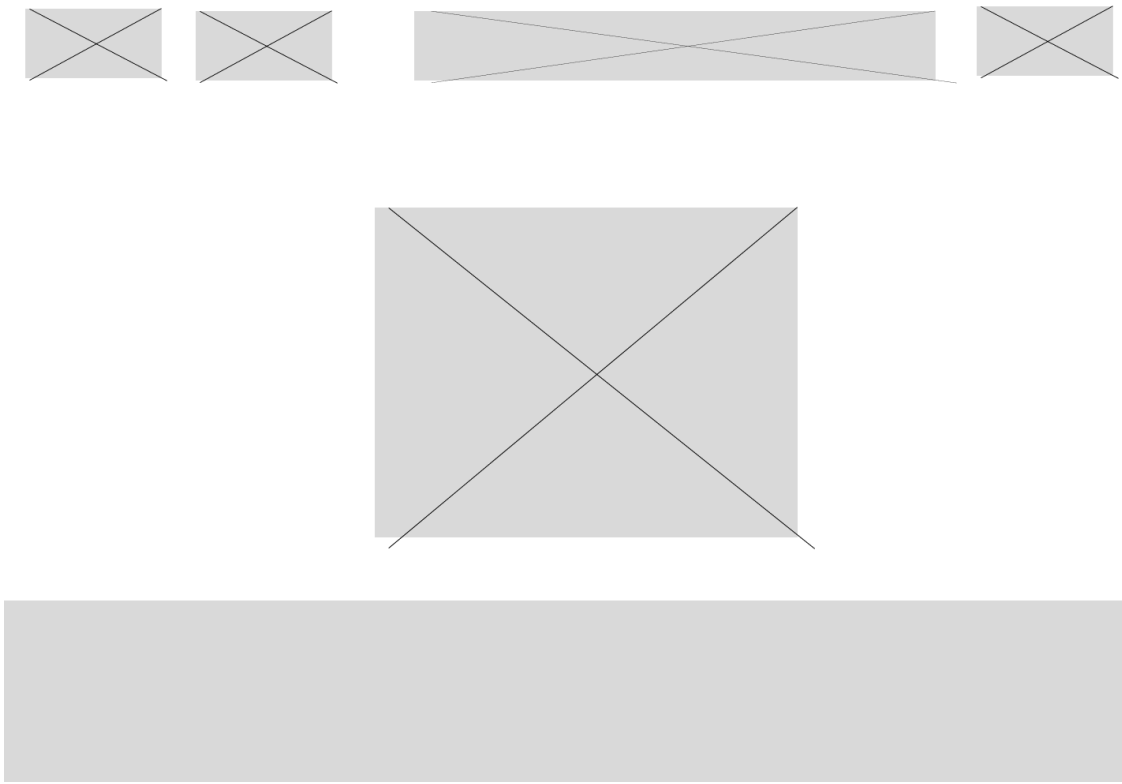


Figura 12 Registrarse (boceto inicial)

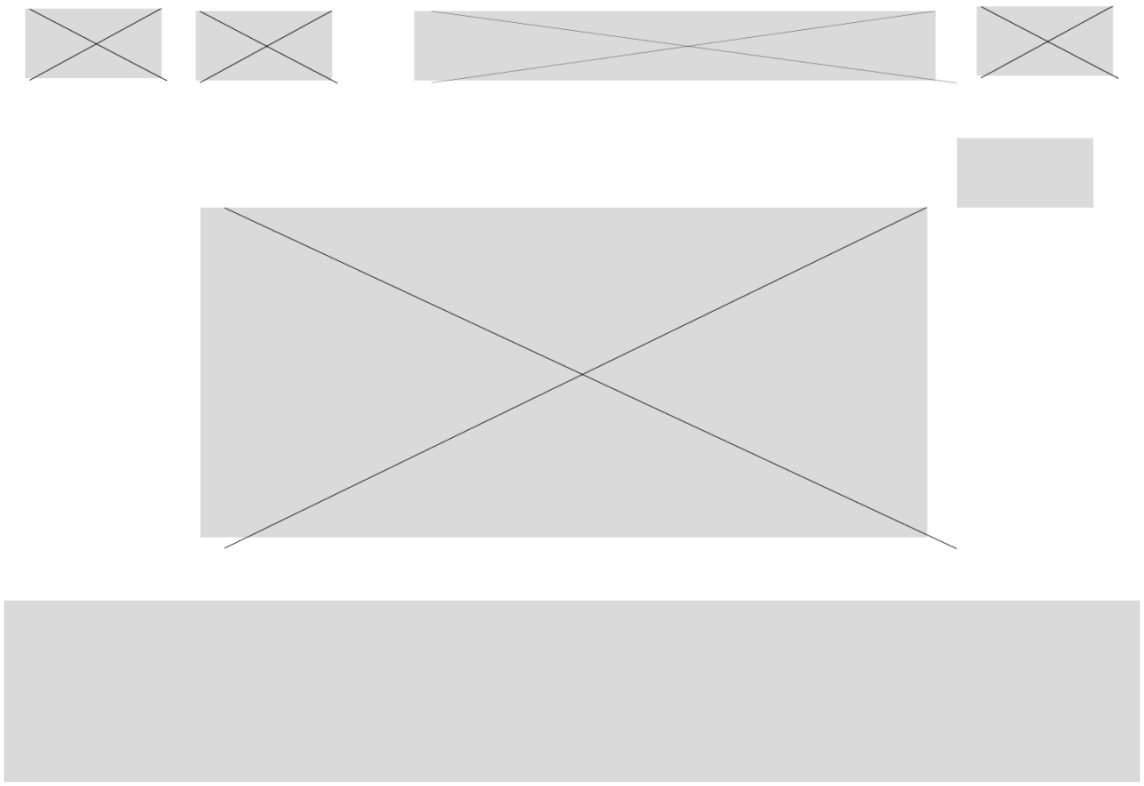


Figura 13 Simulador (boceto inicial)

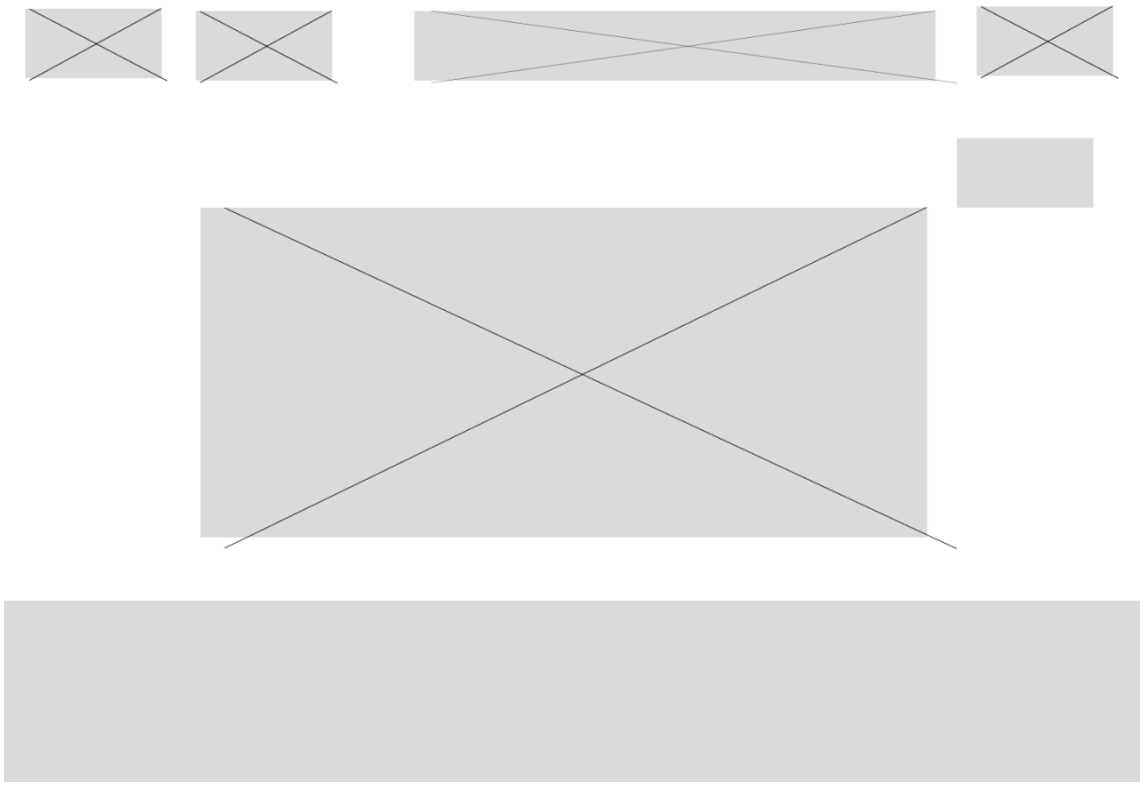


Figura 14 Almacenamiento (boceto inicial)

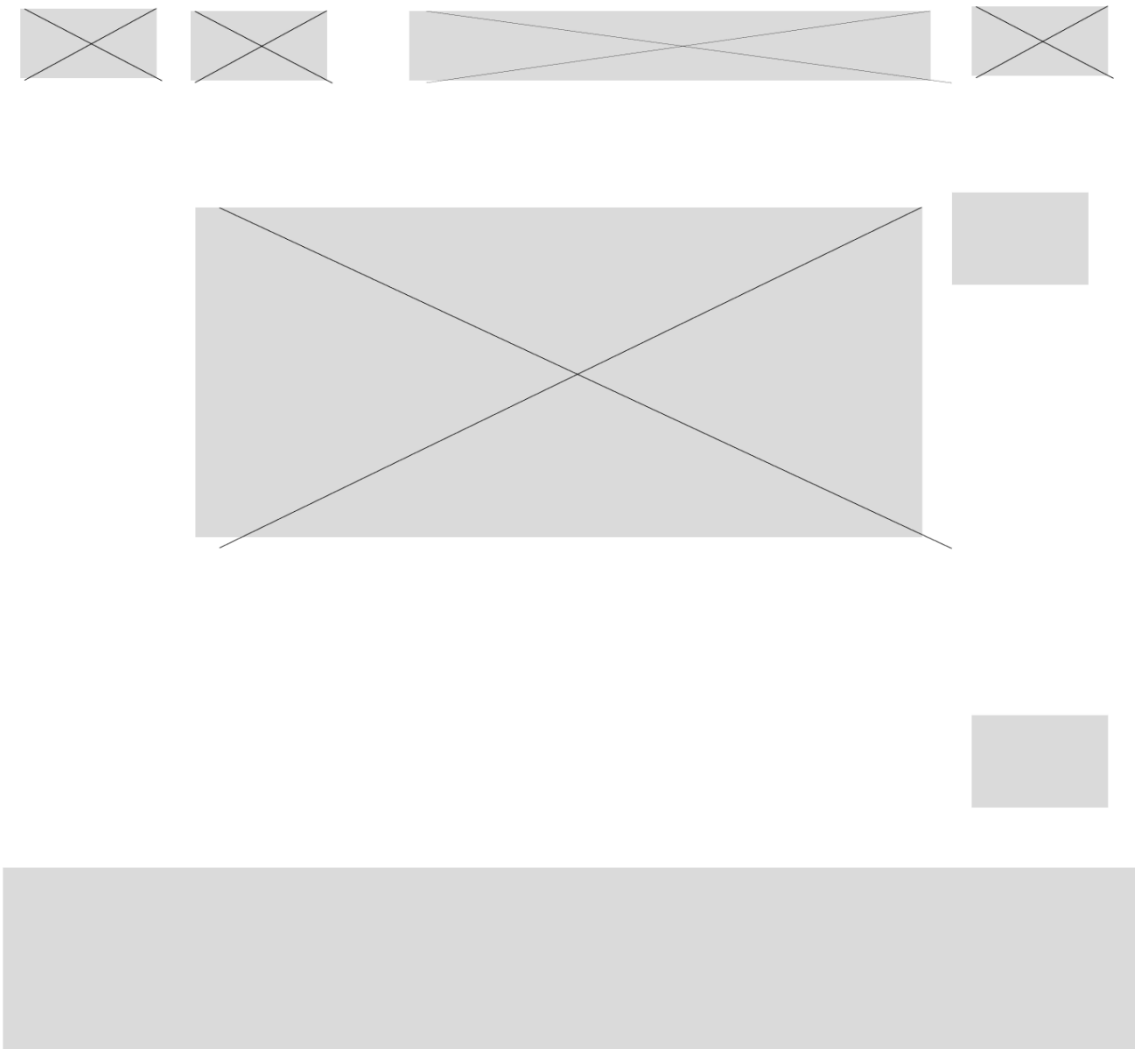


Figura 15 Perfil (boceto inicial)

A continuación, se muestran los mockups finales que reflejan el diseño definitivo de la plataforma.



Figura 16 Simulador (diseño final)

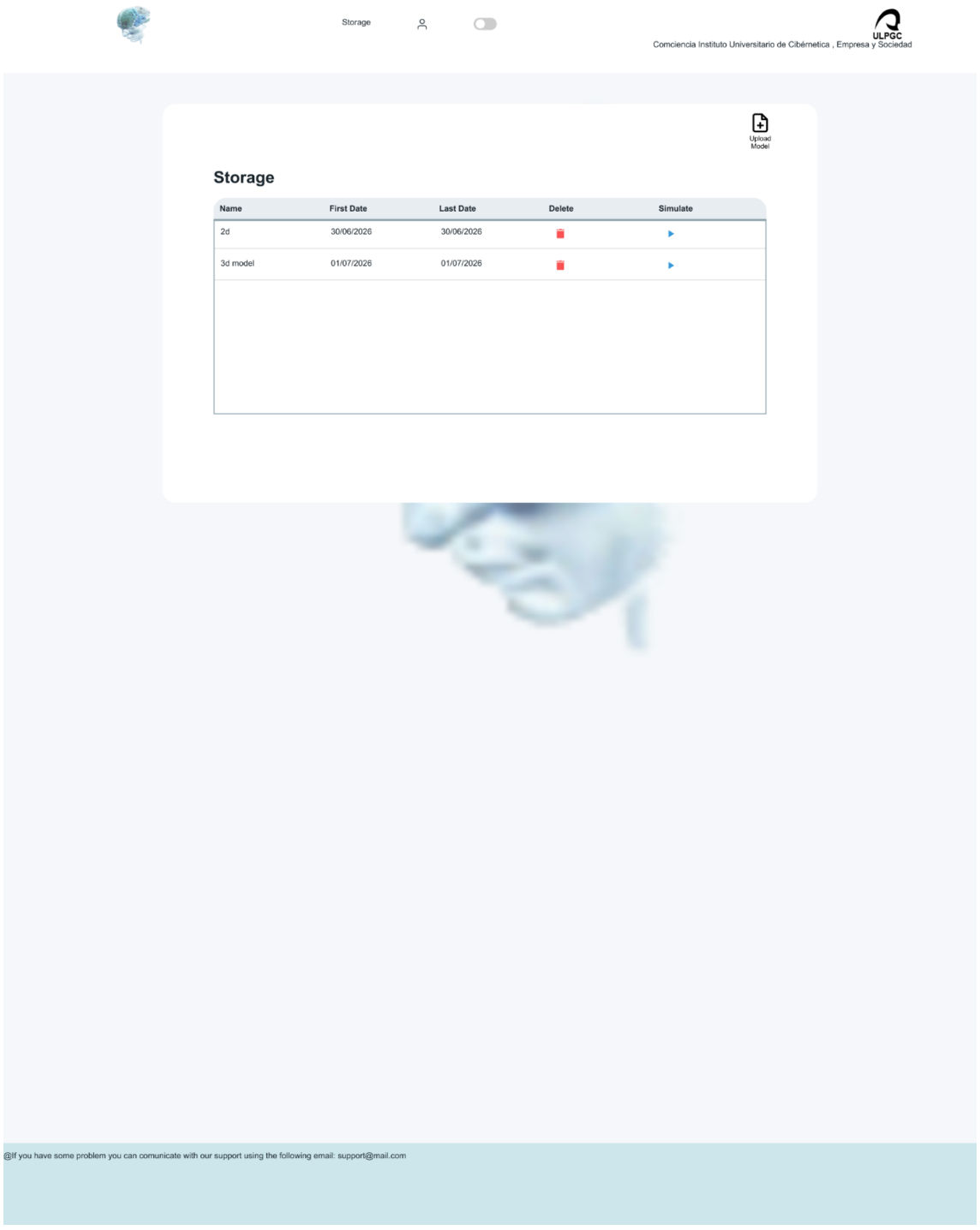


Figura 17 Almacenamiento (diseño final)

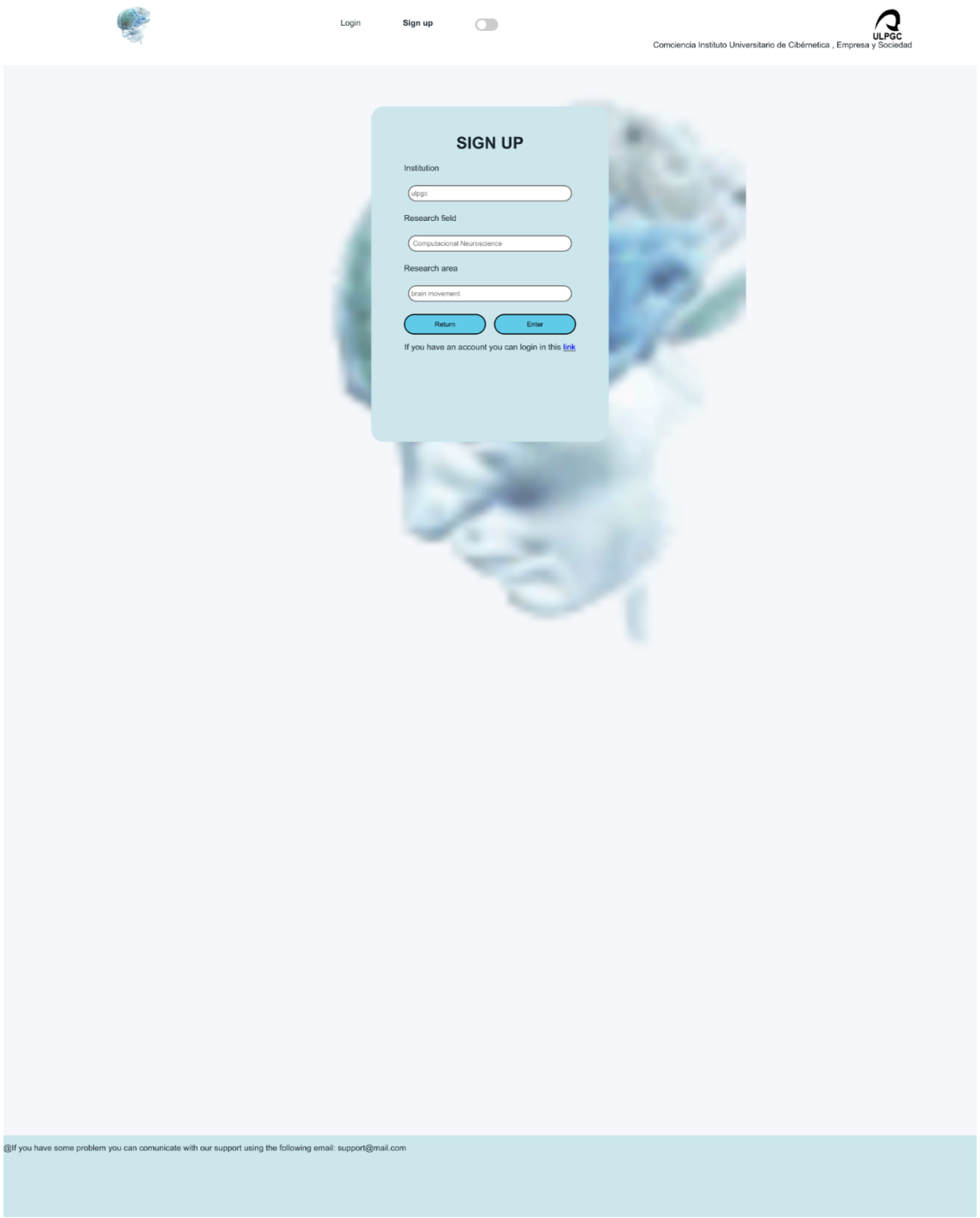


Figura 18 Registrarse (diseño final)

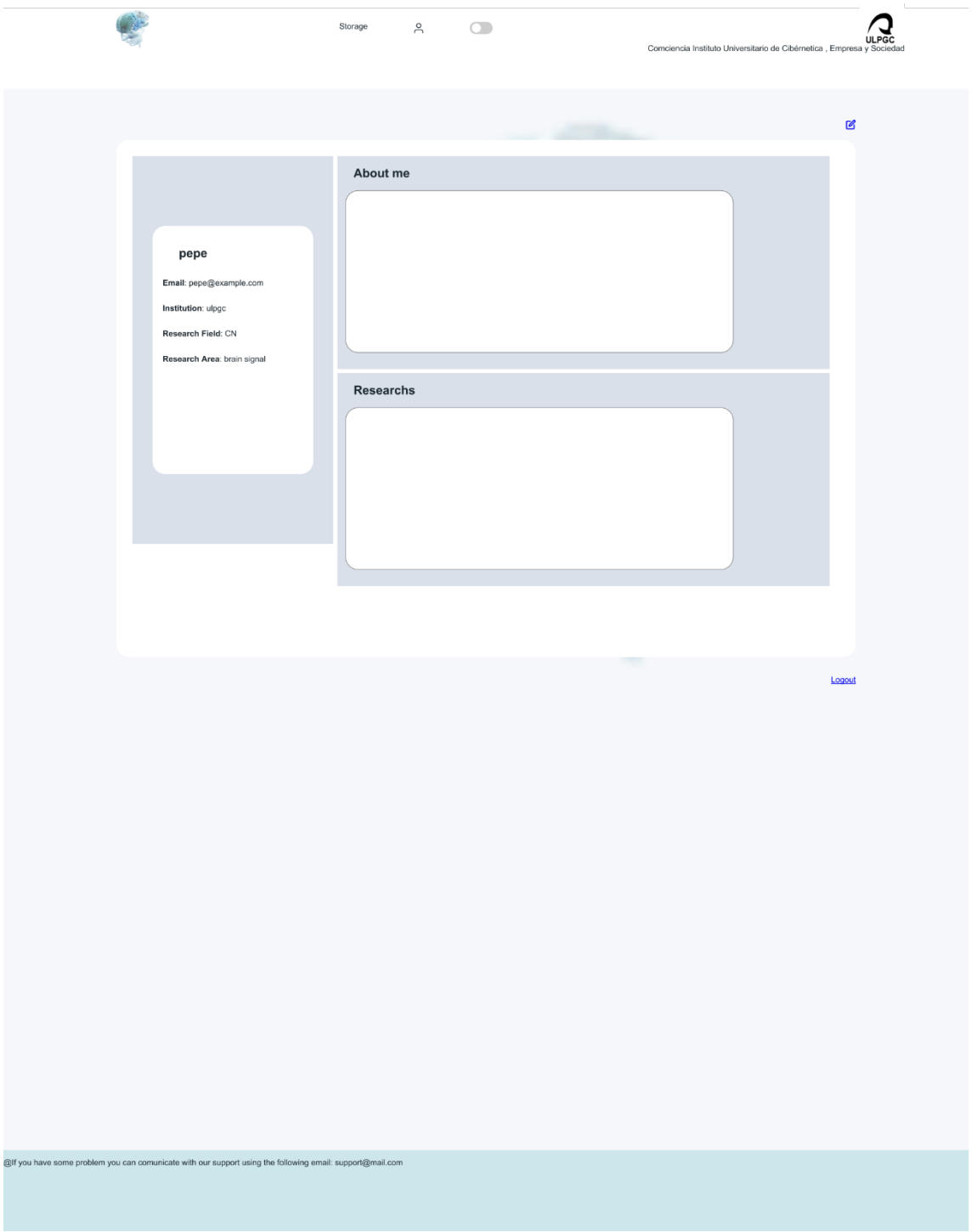


Figura 19 Perfil (diseño final)

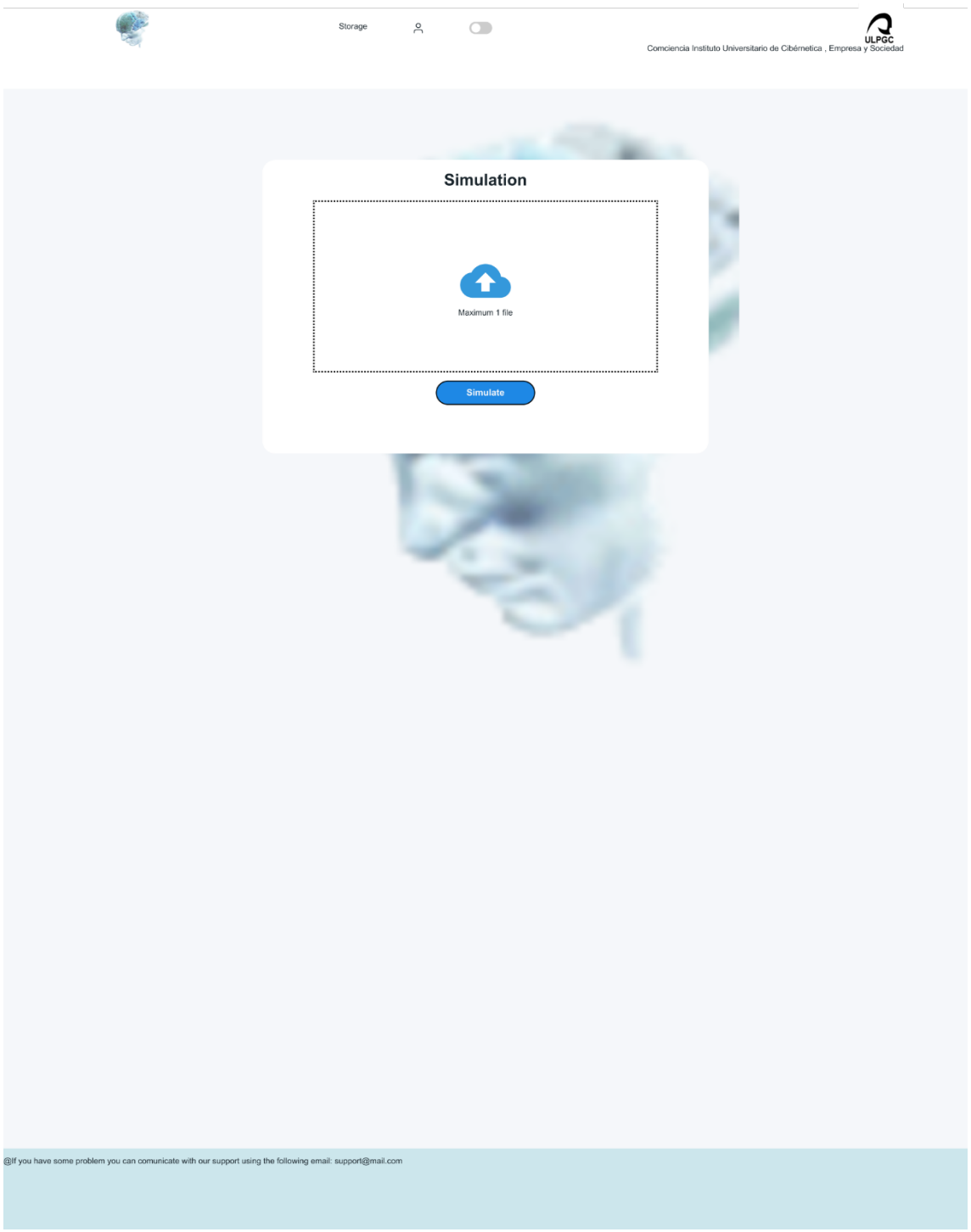


Figura 20 Simulador con resultados (diseño final)

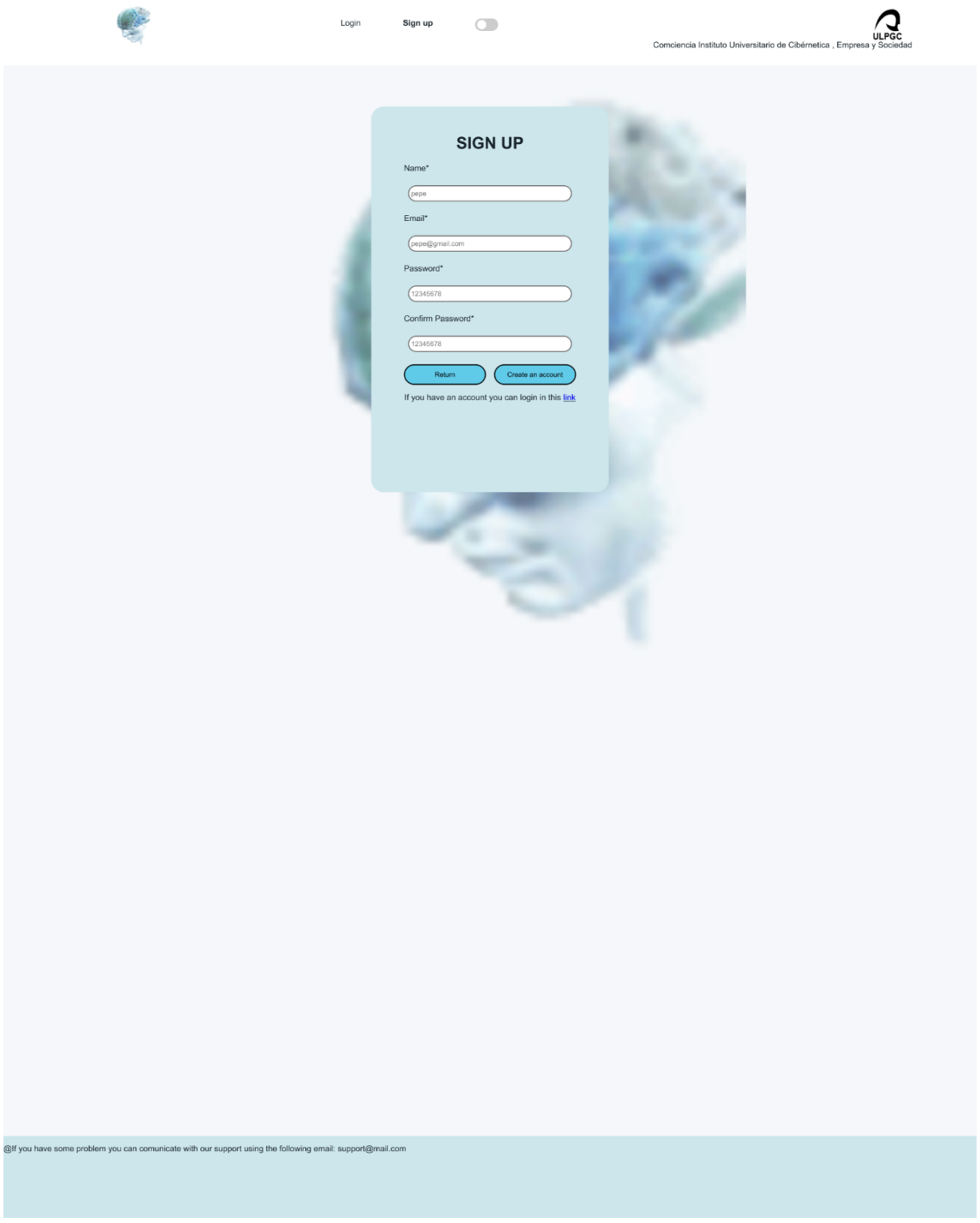


Figura 21 Registrarse con validación (diseño final)

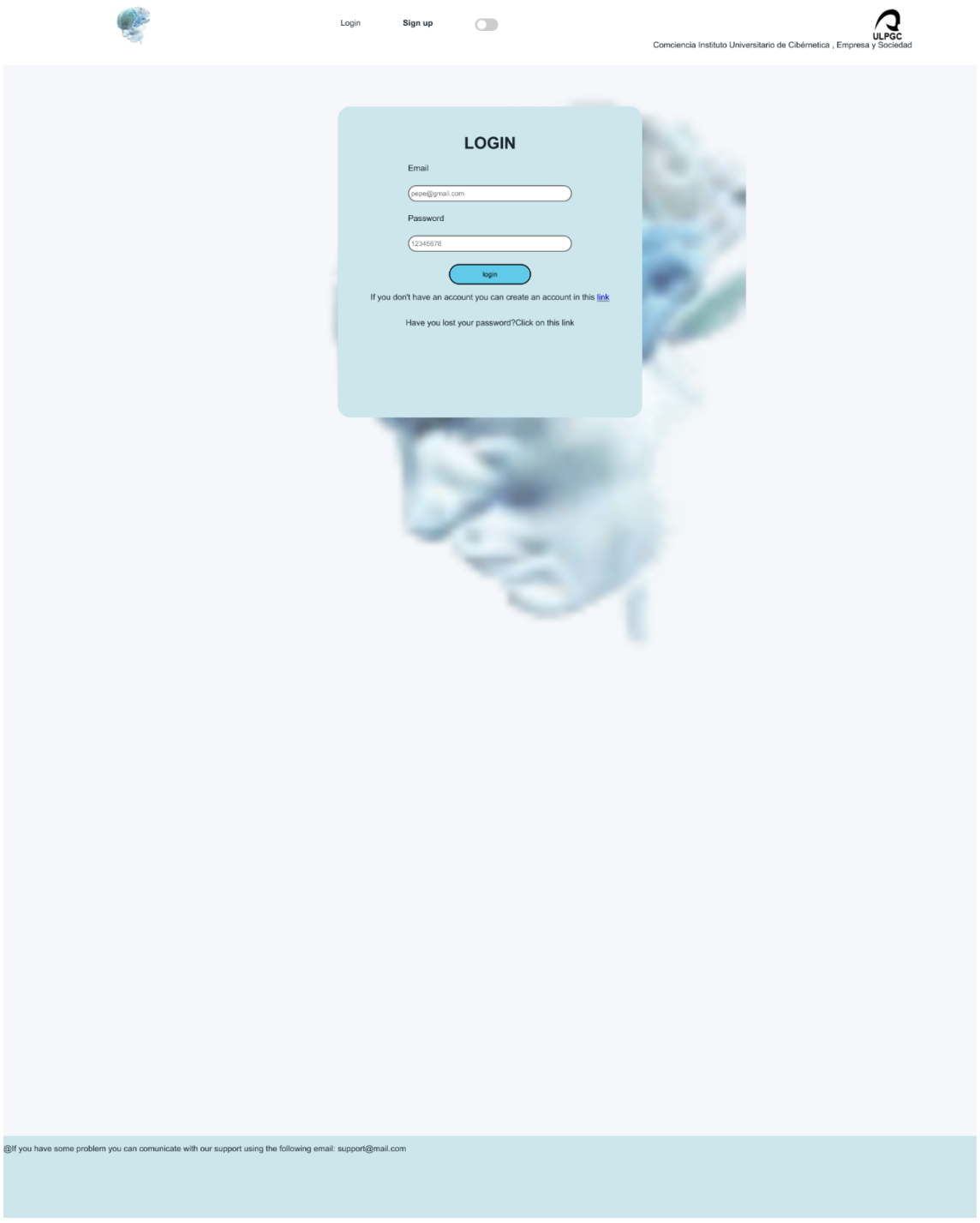


Figura 22 Iniciar sesión (diseño final)



Figura 23 Página de inicio (diseño final)

4.4. Accesibilidad

4.4.1. Ideas de accesibilidad de la plataforma web

La idea principal era hacer la interfaz responsive y aplicarle varios aspectos de accesibilidad como idiomas,modo oscuro,cambio de tamaño de letra y el uso de ARIA en los componentes.

4.4.2. Integraciones de accesibilidad a la plataforma web

Durante el desarrollo de la plataforma hubo bastantes problemas de desarrollo por lo, unicamente se pudo poner los componentes de tipo ARIA y el modo oscuro.

5. Modelos

En este capítulo se muestras las entidades que existen en la aplicación web en el diagrama E/R que muestra sus relaciones

5.1. Diagrama E/R

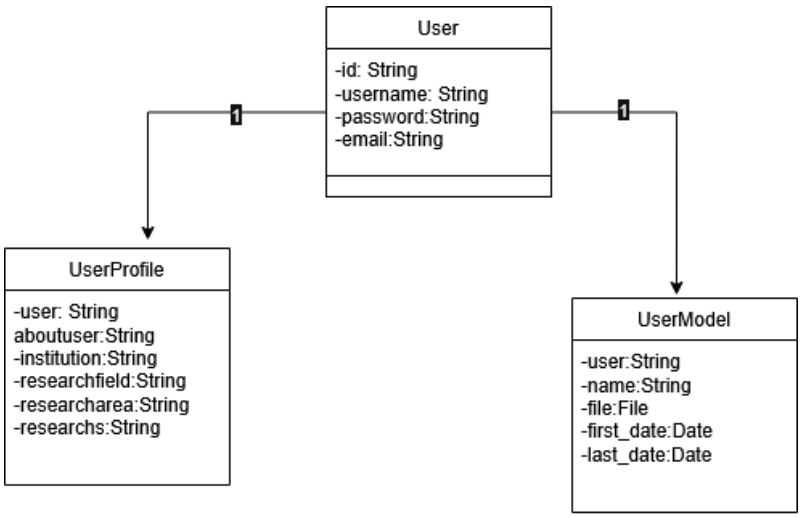


Figura 24 Modelo Entidad-Relación de la base de datos

6. Arquitectura

En este capítulo se explica de forma extensa la parte arquitectónica de la aplicación web, comenzando con el funcionamiento del sistema, los flujos que puede realizar el usuario en la plataforma web y las configuraciones de Docker-compose realizadas que encapsulan la aplicación.

6.1. Arquitectura del sistema

La plataforma sigue una arquitectura cliente-servidor basada en tecnologías web. El usuario accede a la aplicación mediante un navegador web, desde donde interactúa con la interfaz desarrollada utilizando Django. Las peticiones realizadas por el usuario son procesadas por el servidor Django, que ejecuta la lógica de negocio correspondiente y accede a la base de datos PostgreSQL mediante el ORM integrado en el framework. La aplicación se ejecuta dentro de contenedores Docker que permiten aislar los distintos servicios y facilitar su despliegue y mantenimiento. La 25 muestra la arquitectura general del sistema.

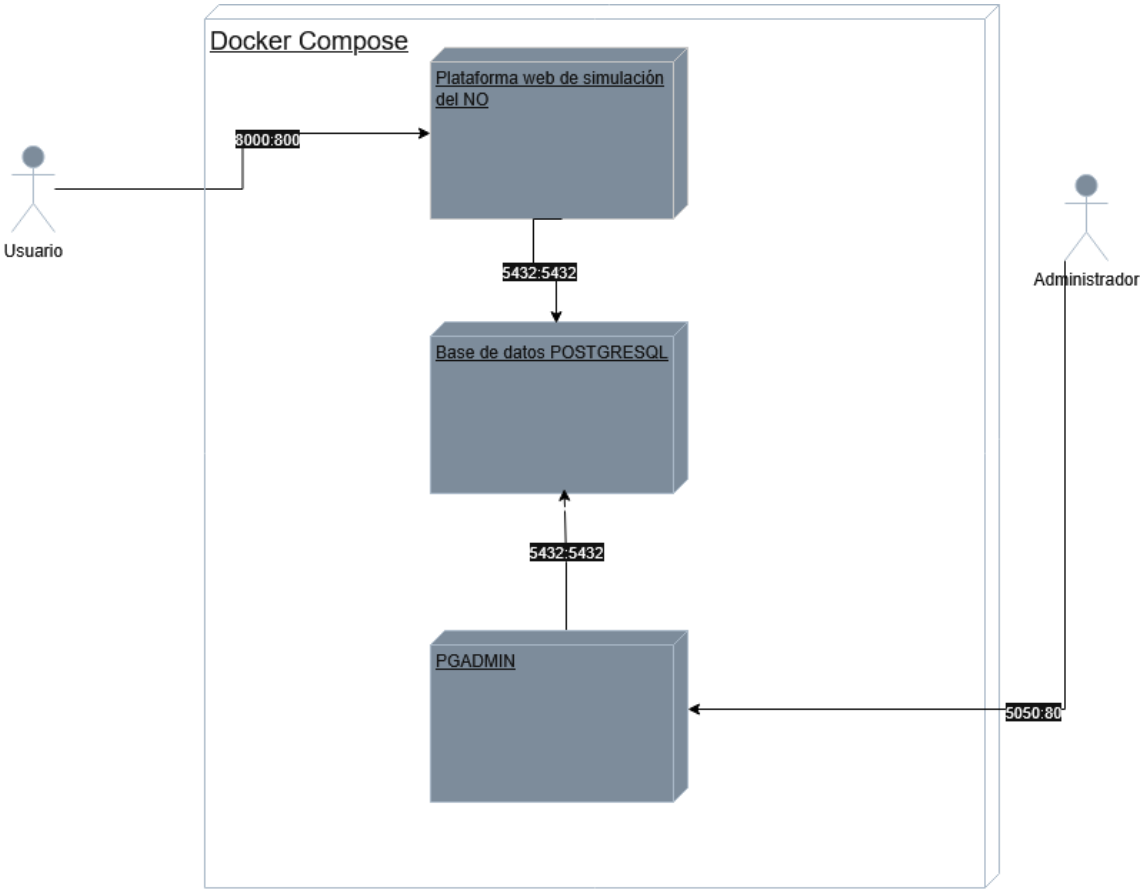


Figura 25 Arquitectura del sistema

Cuadro 8 Componentes principales de la arquitectura

Componentes principales	**Descripción**
Cliente web	Interfaz utilizada por los usuarios para interactuar con la plataforma.
Servidor Django	Encargado de procesar las peticiones y gestionar la lógica de negocio.
PostgreSQL	Sistema gestor de bases de datos encargado del almacenamiento persistente de la información.
pgAdmin	Herramienta de administración de la base de datos.
Docker	Plataforma utilizada para la ejecución y despliegue de los diferentes servicios.

6.2. Flujo funcional de la aplicación

Registro de usuarios

El proceso de registro comienza cuando el usuario selecciona la opción de creación de cuenta disponible en la página principal. A continuación, introduce los datos básicos de autenticación, incluyendo nombre, correo electrónico y contraseña.

Posteriormente, la aplicación solicita información adicional relacionada con la institución, el campo de investigación y el área de especialización. Una vez completado el proceso, el usuario puede acceder a la plataforma utilizando sus credenciales.

Inicio de sesión

Los usuarios registrados pueden acceder a la plataforma mediante su correo electrónico y contraseña. Tras la autenticación correcta, el sistema redirige al usuario a la zona principal de simulación.

Simulación de modelos

El usuario puede cargar un modelo desde el simulador o seleccionar uno previamente almacenado. Tras iniciar la simulación, la plataforma ejecuta el modelo y genera un panel de resultados donde se muestran los parámetros de entrada y las gráficas correspondientes a los resultados obtenidos. Estas gráficas representan la evolución temporal de las concentraciones calculadas por el modelo a lo largo del intervalo de simulación.

Gestión del almacenamiento

La sección de almacenamiento permite gestionar los modelos disponibles en la plataforma. Las operaciones disponibles son: añadir modelos, eliminar modelos y ejecutar simulaciones a partir de modelos almacenados. Para evitar errores accidentales, las operaciones críticas muestran ventanas de confirmación antes de realizar cambios permanentes.

Gestión del perfil

La plataforma incorpora una sección de perfil desde la que el usuario puede modificar información personal como: nombre, institución, campo de investigación, área de investigación, descripción personal, área y línea de investigación. Asimismo, desde esta sección es posible cerrar la sesión activa.

6.3. Configuración de Docker para la base de datos

La base de datos PostgreSQL se despliega mediante Docker Compose utilizando un servicio específico denominado `db`.

Cuadro 9 Configuración de Docker para PostgreSQL

Parámetro	**Descripción**
Imagen	PostgreSQL versión 15
Nombre del contenedor	Identificador único del servicio
Restart	Reinicio automático en caso de fallo
Variables de entorno	Nombre de la base de datos, usuario, contraseña y puerto
Volúmenes	Persistencia de datos
Red	Comunicación con el resto de los contenedores mediante la red virtual del proyecto
Healthcheck	Verificación periódica del estado de la base de datos

```yaml
services:

  db:
    image: postgres:15
    container_name: tfg_db
    restart: always
    environment:
      POSTGRES_DB: ${NAME}
      POSTGRES_USER: ${USER}
      POSTGRES_PASSWORD: ${PASSWORD_D}
    ports:
      - "${PORT:-5432}:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - tfg_network
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${USER} -d ${NAME}"]
      interval: 10s
      timeout: 5s
      retries: 5
```

Figura 26 Configuración Docker de la base de datos

Para la administración de la base de datos se utiliza un segundo servicio basado en pgAdmin.

Cuadro 10 Configuración de Docker para pgAdmin

Parámetro	**Descripción**
Imagen	pgAdmin 4
Nombre del contenedor	Identificador del servicio
Restart	Reinicio automático en caso de fallo
Variables de entorno	Credenciales de acceso
Volúmenes	Puerto 5050
Red	Dependencia del servicio PostgreSQL
Healthcheck	Persistencia de configuración
-	Comunicación con el resto de los servicios

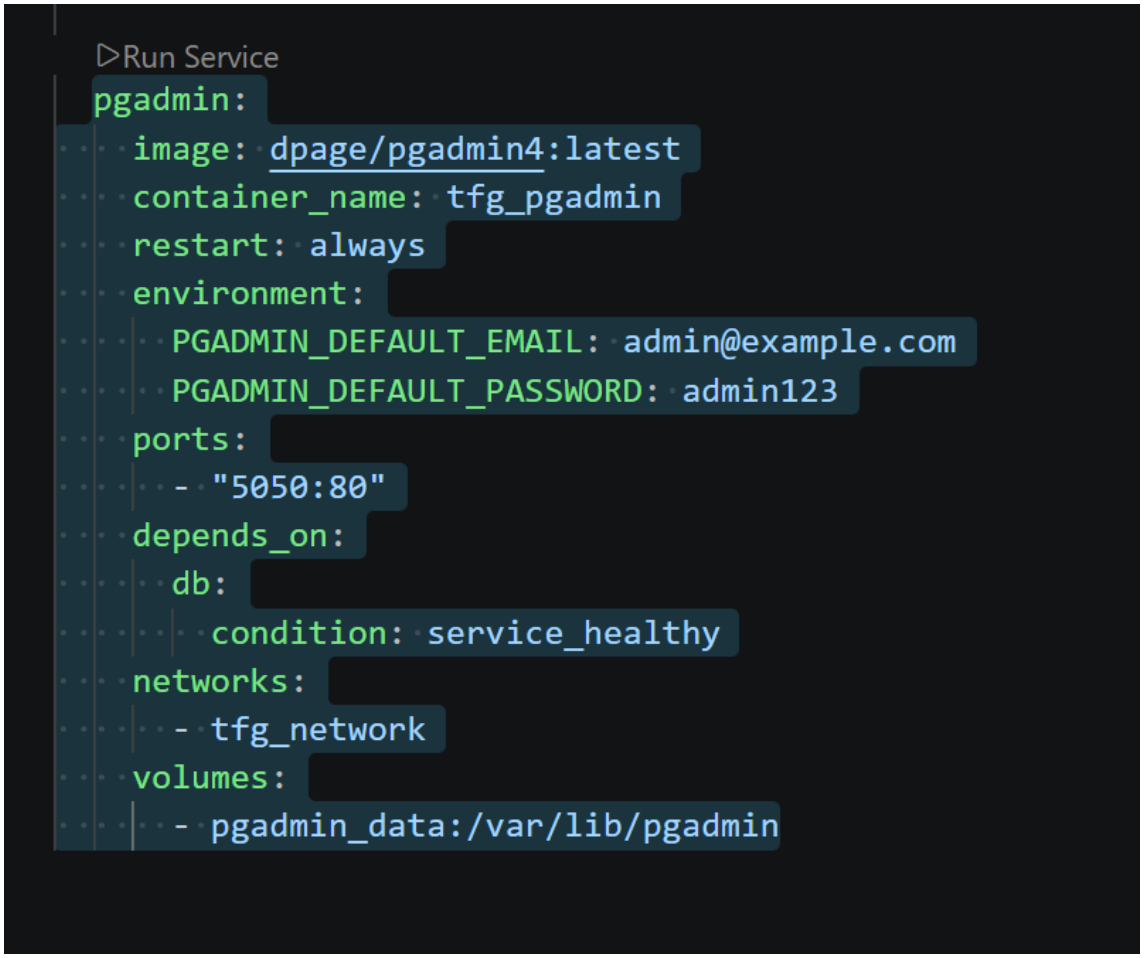


Figura 27 Configuración Docker de la interfaz de base de datos

6.4. Configuración de Docker para la aplicación

La aplicación web también se ejecuta mediante Docker.

Cuadro 11 Configuración de Docker para la aplicación web

Parámetro	**Descripción**
Build	Construcción de la imagen Docker
Nombre del contenedor	Identificador del servicio
Restart	Reinicio automático
Command	Comando de ejecución de Django
Puerto	Puerto de exposición de la aplicación
Depends On	Dependencia del servicio PostgreSQL
Variables de entorno	Configuración de la base de datos y entorno de ejecución
Volúmenes	Compartición de archivos entre el sistema y el contenedor
Red	Comunicación con el resto de los servicios

```
▷Run Service
web:
  build: .
  container_name: tfg_web
  restart: always
  command: python manage.py runserver 0.0.0.0:8000
  ports:
    - "8000:8000"
  depends_on:
    db:
      condition: service_healthy
  environment:
    - DATABASE_URL=postgresql://${USER}:${PASSWORD_D}@db:${PORT:-5432}/${NAME}
    - DEBUG=True
  volumes:
    - .:/app
    - /var/run/docker.sock:/var/run/docker.sock
  networks:
    - tfg_network
```

Figura 28 Configuración Docker para el contenedor de la aplicación web

6.5. Seguridad de la Arquitectura

6.5.1. Ideas de mejora de la seguridad

La idea principal en este proyecto era que, cuando el usuario subiera su modelo o seleccionara un modelo guardado y lo ejecutara, los resultados de dicho modelo no se almacenaran en el disco de los ordenadores de los usuarios. Esto ha sido un reto considerable, ya que la mayoría de las acciones se realizan en disco, por lo que se tuvo que realizar una investigación durante varias semanas para comprender cómo hacerlo posible. También se tuvieron en cuenta las etiquetas exigidas por el propio framework de Django para posibles ataques XSS, añadiendo en cada formulario un etiquetado específico para poder ejecutarlo de forma segura.

6.5.2. Investigación para mejorar la seguridad de la plataforma web

Las investigaciones fueron bastante avanzadas, pensando en cómo elaborar un sandbox, que es básicamente una caja negra donde se ejecuta código de cualquier tipo sin preocuparse de que puedan atacar el propio sistema, ya que está aislado. Por lo que se comenzó primero haciendo que todos los códigos se ejecutaran en entornos virtuales de Python. Sin embargo, al observar el administrador de tareas, se comprobó que este cargaba en el disco, además de confirmarlo en otras fuentes de internet, por lo que se descartó la idea.

Otra posible solución era ejecutarlo en una plataforma como AWS, Microsoft Azure o Google Cloud Platform. Sin embargo, esta opción también se descartó por falta de presupuesto. Observando tutoriales, se vio la posibilidad de ejecutar los códigos en RAM a través de Docker. Aunque comparte el núcleo del sistema operativo que lo ejecuta, existe una capa asociada a la RAM para datos temporales, lo cual se escogió como solución.

6.5.3. Integraciones de la seguridad de la plataforma web

Con la idea en mente de añadir Docker para la ejecución de código que pudiera ser dañino o no, se examinaron las posibles formas de integrarlo en el desarrollo. Por ello, se investigó la librería

`subprocess` para ejecutar de forma automatizada el comando que generara la imagen en el caso de que no existiera, y entonces crear el contenedor añadiendo el modelo y ejecutándolo en la capa de la RAM, sin añadir volúmenes en Docker para que no compartiera el sistema de ficheros y no quedara constancia en el disco. En la imagen se definirían las librerías matemáticas y científicas que podrían usar los modelos.

7. Desarrollo del simulador de los modelos de difusión del NO

El simulador se dividió en dos fases: la parte donde se sube o se selecciona el modelo que se va a simular, y después la representación gráfica. Sin embargo, no se ha hablado internamente de cómo funciona, y es fundamental ya que es el corazón del proyecto.

Se empezará primero con el fragmento de código donde se interpreta cómo llega el modelo al simulador, si vía GET (eso significa que está dentro de nuestro sistema) o vía POST (que significa que no está dentro de nuestro sistema).

GET

Vía GET, como se habló previamente, significa que viene dentro de nuestro sistema, por lo que se toma de la sesión el ID del modelo pasado por la función de la vista de almacenamiento y se comprueba si esa persona está enlazada de verdad con ese modelo. Tras confirmar que es de esa persona, se lee el contenido del modelo y se comprueba si es de Python, añadiéndolo luego a la lista de ficheros y eliminándolo de la sesión.

POST

Vía POST, básicamente se recoge el fichero que ha dado el usuario, se comprueba que es de tipo Python y que el tamaño no supera cierto umbral, y se añade a la lista de ficheros.

La idea de hacer una lista de ficheros es dejar la posibilidad más adelante de que cualquier desarrollador pueda añadir más de un fichero. En este proyecto se ha decidido hacer con uno por facilidad de desarrollo y por escalabilidad del sistema.

```python
def home(request):
    if request.method == "GET" and 'simulacion_completa' in request.session:
        del request.session['simulacion_completa']

    files = []
    if request.method == "GET" and 'archivo_modelo_id' in request.session:
        modelo_id = request.session.get('archivo_modelo_id')
        instancia_modelo = get_object_or_404(UserModel, id=modelo_id, user=request.user)
        archivo_emulado = ContentFile(instancia_modelo.file.read())
        nombre_seguro = instancia_modelo.name.replace(" ", "_")
        archivo_emulado.name = nombre_seguro if nombre_seguro.endswith('.py') else f"{nombre_seguro}.py"
        files.append(archivo_emulado)
        del request.session['archivo_modelo_id']

    elif request.method == "POST" and request.FILES.get('file'):
        model_file = request.FILES.get('file')
        if model_file.name.endswith('.py') and model_file.size < 2000000:
            files.append(model_file)
```

Figura 29 Código del simulador - Gestión de peticiones (Parte 1)

```
if files:
    container_name = f"simulation_{request.user.id}"
    files[0].seek(0)
    code = files[0].read().decode('utf-8')

    setup_numpy = (
        "import numpy as np\n"
        "try:\n"
        "    np.set_printoptions(threshold=np.inf)\n"
        "except:\n"
        "    pass\n"
    )

    extractor_json = (
        "\n\n"
        "import json\n"
        "import types\n"
        "import numpy as np\n"
        "valores_finales = {} \n"
        "for k, v in list(globals().items()):\n"
        "    if k.startswith('_') or k in ['json', 'types', 'np', 'valores_finales', 'k', 'v']:\n"
```

Figura 30 Código del simulador - Gestión de peticiones (Parte 2)

```
        "for k, v in list(globals().items()):\n"
        "    if k.startswith('_') or k in ['json', 'types', 'np', 'valores_finales', 'k', 'v']:\n"
        "        continue\n"
        "    if isinstance(v, (types.ModuleType, types.FunctionType, types.BuiltinFunctionType)):\n"
        "        continue\n"
        "    try:\n"
        "        if isinstance(v, np.ndarray):\n"
        "            valores_finales[k] = v.tolist()\n"
        "        else:\n"
        "            json.dumps(v)\n"
        "            valores_finales[k] = v\n"
        "    except:\n"
        "        valores_finales[k] = str(v)\n"
        "\n"
        "import sys\n"
        "sys.stdout.write('==JSON_START==' + json.dumps(valores_finales) + '==JSON_END==')\n"
    )

    codigo_final = setup_numpy + code + extractor_json
```

Figura 31 Código del simulador - Gestión de peticiones (Parte 3)

Tras pasar los condicionales para saber el tipo de petición, se comprueba si la lista de ficheros tiene ficheros. Si los tiene, se procede a establecer el nombre del contenedor e inicializar el puntero del fichero, para luego leer el contenido del fichero en el formato UTF-8. Después se realiza una configuración de NumPy para que no recorte los datos (ya que puede suceder que sea tan grande que los recorte) y otra configuración para extraer las variables de entrada y salida del modelo, eliminando las variables con ciertas características como las que empiezan por `_` o si es una función o método, transformándolas todas luego a un formato JSON. Finalmente, en la variable `codigo_final` se almacena todo para la siguiente parte.



```
    try:
        subprocess.run(["docker", "rm", "-f", container_name], capture_output=True)

        resultado = subprocess.run([
            "docker", "run", "--rm", "-i",
            "--name", container_name,
            "--network", "none",
            "container", "python"
        ], input=codigo_final, capture_output=True, text=True, timeout=15)

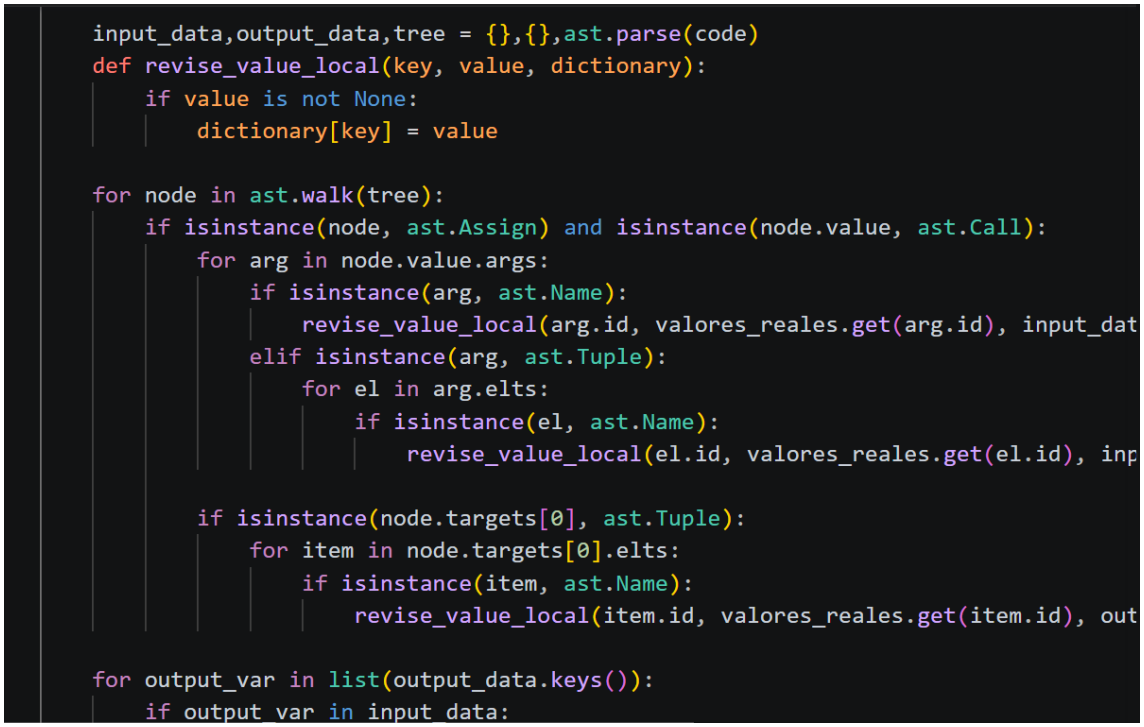
        valores_reales = {}
        if resultado.returncode == 0:
            stdout_data = resultado.stdout
            if "==JSON_START==" in stdout_data and "==JSON_END==" in stdout_data:
                try:
                    json_puro = stdout_data.split("==JSON_START==")[1].split("==JSON_END==")[0].strip()
                    valores_reales = json.loads(json_puro)
                except json.JSONDecodeError:
                    pass
        else:
            error_script = resultado.stderr if resultado.stderr else "Error de ejecución."
            return render(request, 'simulator/home.html', {'error': f"Error en tu script: {error_script}"})
```

Figura 32 Código del simulador - Ejecución en contenedor (Parte 4)

Este bloque de código realiza la ejecución segura y el análisis del script de Python que subió el usuario dentro de un entorno aislado.

Primero, el sistema intenta borrar cualquier contenedor previo con el mismo nombre para evitar conflictos de ejecución. Acto seguido, lanza un contenedor de Docker temporal configurado de forma muy estricta por motivos de seguridad, ya que le apaga por completo el acceso a Internet para evitar ataques informáticos y le impone un límite de tiempo estricto de quince segundos para que el servidor no se cuelgue si el script tiene un bucle infinito. El código modificado del usuario se inyecta directamente al contenedor a través de su entrada estándar.

Al terminar la ejecución, el sistema evalúa el código de salida del contenedor. Si el programa corrió exitosamente, el sistema lee la consola del contenedor, localiza las etiquetas especiales llamadas `JSON START` y `JSON END` para recortar el texto intermedio, y transforma ese JSON de vuelta en un diccionario de variables nativo de Python listo para ser procesado. Si el script del usuario falla por algún error de sintaxis o lógica, el sistema captura el mensaje de error exacto de la consola de Python y recarga la página web para mostrarle al usuario el fallo exacto para que pueda corregirlo.



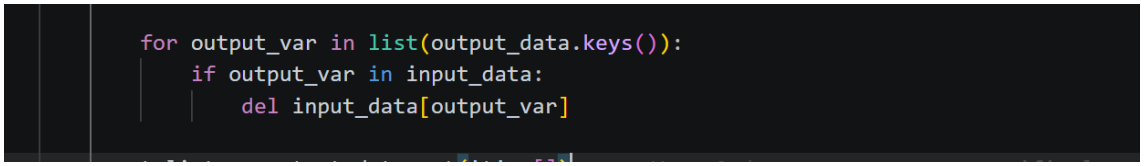
```
input_data,output_data,tree = {},{},ast.parse(code)
def revise_value_local(key, value, dictionary):
    if value is not None:
        dictionary[key] = value

for node in ast.walk(tree):
    if isinstance(node, ast.Assign) and isinstance(node.value, ast.Call):
        for arg in node.value.args:
            if isinstance(arg, ast.Name):
                revise_value_local(arg.id, valores_reales.get(arg.id), input_dat
            elif isinstance(arg, ast.Tuple):
                for el in arg.elts:
                    if isinstance(el, ast.Name):
                        revise_value_local(el.id, valores_reales.get(el.id), inp

        if isinstance(node.targets[0], ast.Tuple):
            for item in node.targets[0].elts:
                if isinstance(item, ast.Name):
                    revise_value_local(item.id, valores_reales.get(item.id), out

for output_var in list(output_data.keys()):
    if output_var in input_data:
```

Figura 33 Código del simulador - Procesamiento de resultados (Parte 5)



```
for output_var in list(output_data.keys()):
    if output_var in input_data:
        del input_data[output_var]
```

Figura 34 Código del simulador - Procesamiento de resultados (Parte 6)

Este bloque de código analiza la estructura interna del script de Python del usuario para clasificar automáticamente cuáles variables actuaron como entradas y cuáles como salidas en las funciones de simulación.

Primero, el sistema inicializa dos diccionarios vacíos para almacenar estos datos y utiliza la librería AST para convertir el código de texto en un árbol de nodos abstractos que representa su estructura lógica. También define una función auxiliar interna que se encarga de verificar que una variable tenga un valor real asignado antes de guardarla en el diccionario correspondiente. Acto seguido, el sistema inicia un recorrido por todos los componentes del árbol, buscando específicamente aquellas líneas donde ocurre una asignación de variables que involucre el llamado a una función.

Dentro de estas líneas detectadas, el sistema inspecciona los argumentos pasados a la función, examinando si son variables directas o si están agrupadas dentro de una tupla para extraer sus nombres y cruzarlos con los datos reales calculados previamente en el contenedor, guardándolos en el diccionario de entradas. Al mismo tiempo, el sistema revisa la parte izquierda de la asignación para identificar qué variables están recibiendo el resultado de esa función. Si detecta que se están desempaquetando en una tupla, extrae cada uno de sus nombres y los almacena en el diccionario de salidas junto a sus valores finales. Para concluir el proceso, el sistema ejecuta una limpieza final comparando ambos diccionarios y eliminando del grupo de entradas cualquier variable que también haya sido detectada como una salida, evitando duplicados para que la información quede perfectamente clasificada.

```
    t_lista = output_data.get('t', [])
    c_valores = output_data.get('C', [])

    json_unificado = {
        "initial_data": input_data,
        "final_results": {      You, 2 days ago • proceso semifinal
            "time": t_lista,
            "concentration": c_valores
        }
    }

    request.session['modelo']= code
    request.session['simulacion_completa'] = json_unificado
    return redirect('simulator')
```

Figura 35 Código del simulador - Almacenamiento en sesión (Parte 7)

Este bloque de código se encarga de estructurar y empaquetar los datos finales obtenidos de la simulación para almacenarlos en la sesión del usuario y redirigirlo a la vista principal.

Después de la parte lógica, se explicará la lógica para la visualización.

```
document.addEventListener("DOMContentLoaded", function() {
    try {
        const el = document.getElementById("datos-django");
        if (!el || !el.textContent.trim()) return;

        const paquete = JSON.parse(el.textContent.trim());
        const t_axis = paquete.final_results.time || [];
        const concentration = paquete.final_results.concentration || [];
        const initialData = paquete.initial_data || {};
        const dataShape = getArrayShape(concentration);
        const rows = Number(initialData.rowN_ || initialData.N_ || dataShape[0] || 1);
        const cols = Number(initialData.columnN_ || (dataShape.length >= 3 ? dataShape[1] : 1) || 1);
        const depths = Number(initialData.depthN_ || (dataShape.length >= 4 ? dataShape[2] : 1) || 1);
        const pasos_t = t_axis.length;
```

Figura 36 Código del simulador - Inicialización de visualización (1)

Este primer fragmento se activa automáticamente cuando la página web termina de cargarse por completo en el navegador. Su propósito es actuar como el puente de datos entre el servidor y la interfaz gráfica, buscando un elemento HTML específico que esconde la información de la simulación en formato de texto plano. Una vez localizado el contenedor, el sistema convierte ese texto crudo en un objeto estructurado de JavaScript, del cual extrae los vectores de tiempo, concentración y las variables de configuración inicial. Con estos datos, el script calcula el volumen del espacio simulado, determinando con precisión matemática el número exacto de filas, columnas y capas de profundidad que componen la estructura de la simulación, junto a la cantidad total de pasos temporales registrados.

```
const title = document.getElementById("dynamic-title");
const linealChartEl = document.getElementById("lineal_chart");
const scatterChartEl = document.getElementById("scatter_chart");
const heatmapChartEl = document.getElementById("heatmap_chart");
const volumeChartEl = document.getElementById("volume_chart");

if (document.getElementById("rows")) document.getElementById("rows").textContent = rows;
if (document.getElementById("cols")) document.getElementById("cols").textContent = cols;
if (document.getElementById("depths")) document.getElementById("depths").textContent = depths;
if (document.getElementById("compartimentals")) document.getElementById("compartimentals").textContent = rows * cols * depths;
if (document.getElementById("start_t")) document.getElementById("start_t").textContent = `${initialData.start_t ?? ""} s`;
if (document.getElementById("end_t")) document.getElementById("end_t").textContent = `${initialData.end_t ?? ""} s`;
if (document.getElementById("delta_t")) document.getElementById("delta_t").textContent = `${initialData.delta_t ?? ""} s`;
You, 14 hours ago • Uncommitted changes
if (title) title.innerText = "Simulation";

setDisplay(linealChartEl, false);
setDisplay(scatterChartEl, true);
setDisplay(heatmapChartEl, true);
setDisplay(volumeChartEl, true);

function setDisplay(element, visible) {
    if (element) element.style.display = visible ? "block" : "none";
}
```

Figura 37 Código del simulador - Configuración de interfaz (2)

Este segmento se encarga de preparar la parte visual de la página web, enlazando las variables lógicas con los elementos del documento. El sistema busca los contenedores HTML destinados a mostrar los textos informativos y escribe en ellos los valores calculados de dimensiones espaciales, el total de compartimentos resultante de multiplicar filas por columnas y profundidades, así como las métricas físicas de tiempo inicial, tiempo final y los intervalos de cambio de la simulación. Adicionalmente, define una función auxiliar de control de visibilidad que oculta o muestra dinámicamente los lienzos de las gráficas, estableciendo un estado inicial donde por defecto se apaga el gráfico lineal y se encienden los paneles destinados a los gráficos de dispersión, mapas de calor y volúmenes tridimensionales.

```
function getArrayShape(value) {
    const shape = [];
    let current = value;
    while (Array.isArray(current)) {
        shape.push(current.length);
        current = current[0];
    }
    return shape;
}

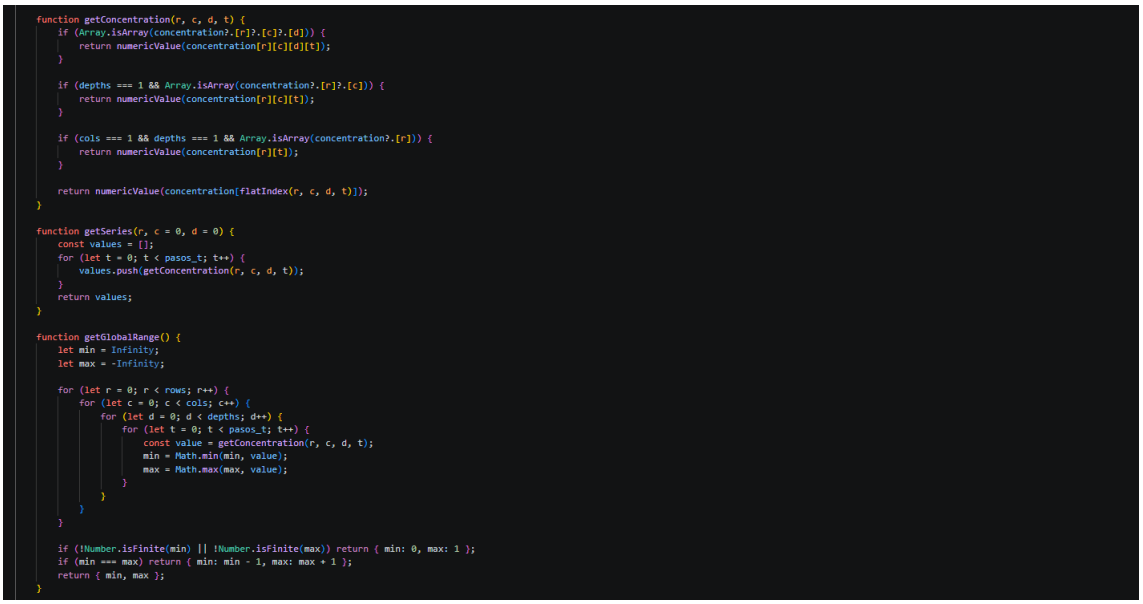
function numericValue(value) {
    const numberValue = Number(value);
    return Number.isFinite(numberValue) ? numberValue : 0;
}

function flatIndex(r, c, d, t) {
    return (((r * cols + c) * depths + d) * pasos_t) + t;
}
```

Figura 38 Código del simulador - Funciones auxiliares (3)

Esta sección declara las herramientas matemáticas necesarias para interpretar la geometría espacial y proteger la integridad de los datos numéricos. La primera función analiza el anidamiento de las matrices de concentración, ejecutando un bucle que mide la longitud de cada dimensión para devolver un arreglo con las proporciones exactas de la estructura de datos. La segunda función actúa como un filtro de seguridad que transforma cualquier dato entrante en un número real válido, sustituyendo valores indefinidos o erróneos por un cero para evitar caídas en el renderizado. La tercera función implementa una fórmula matemática de indexación que mapea coordenadas tridimensionales (fila, columna y profundidad) junto con el paso del tiempo, convirtiendo esa posición lógica en un índice lineal único para cuando los datos vengan empaquetados en un solo

arreglo continuo.



```
function getConcentration(r, c, d, t) {
    if (Array.isArray(concentration?.[r]?.[c]?.[d])) {
        return numericValue(concentration[r][c][d][t]);
    }

    if (depths === 1 && Array.isArray(concentration?.[r]?.[c])) {
        return numericValue(concentration[r][c][t]);
    }

    if (cols === 1 && depths === 1 && Array.isArray(concentration?.[r])) {
        return numericValue(concentration[r][t]);
    }

    return numericValue(concentration[flatIndex(r, c, d, t)]);
}

function getSeries(r, c = 0, d = 0) {
    const values = [];
    for (let t = 0; t < pasos_t; t++) {
        values.push(getConcentration(r, c, d, t));
    }
    return values;
}

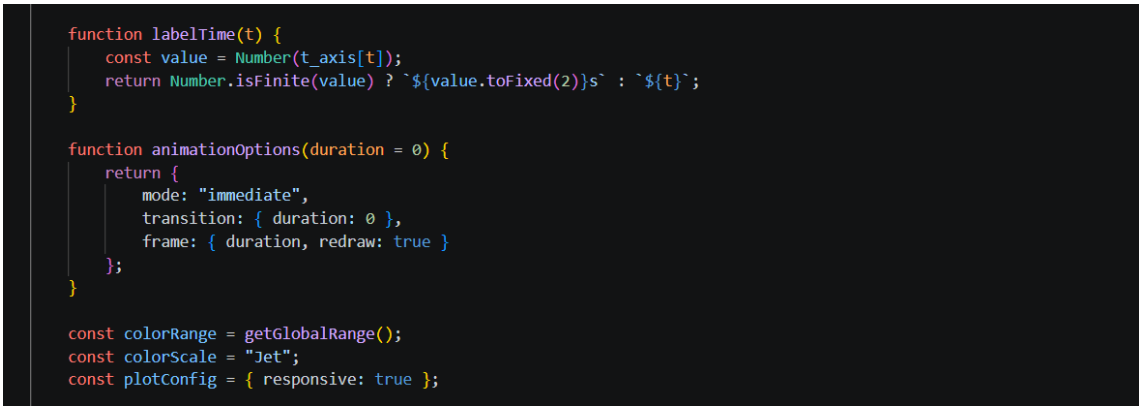
function getGlobalRange() {
    let min = Infinity;
    let max = -Infinity;

    for (let r = 0; r < rows; r++) {
        for (let c = 0; c < cols; c++) {
            for (let d = 0; d < depths; d++) {
                for (let t = 0; t < pasos_t; t++) {
                    const value = getConcentration(r, c, d, t);
                    min = Math.min(min, value);
                    max = Math.max(max, value);
                }
            }
        }
    }

    if (!Number.isFinite(min) || !Number.isFinite(max)) return { min: 0, max: 1 };
    if (min === max) return { min: min - 1, max: max + 1 };
    return { min, max };
}
```

Figura 39 Código del simulador - Extracción de datos (4)

Este bloque concentra la lógica de extracción de información y normalización de escalas visuales del script. La función principal de lectura evalúa de forma condicional la estructura de la matriz de concentración para extraer el dato numérico correcto, adaptándose automáticamente si la simulación se organizó en cuatro, tres, dos o una sola dimensión en base a las coordenadas solicitadas. A su vez, la función de series compila la evolución histórica de una celda específica a lo largo de todos los pasos temporales, guardándola en una lista ordenada. Finalmente, el algoritmo de rango global ejecuta un recorrido exhaustivo por absolutamente cada punto del espacio y momento del tiempo, comparando los valores para hallar la concentración mínima y máxima de toda la simulación, lo cual resulta indispensable para fijar los límites de color de los gráficos y asegurar que los cambios de tonalidad sean coherentes durante las animaciones.



```
function labelTime(t) {
    const value = Number(t_axis[t]);
    return Number.isFinite(value) ? `${value.toFixed(2)}s` : `${t}`;
}

function animationOptions(duration = 0) {
    return {
        mode: "immediate",
        transition: { duration: 0 },
        frame: { duration, redraw: true }
    };
}

const colorRange = getGlobalRange();
const colorScale = "Jet";
const plotConfig = { responsive: true };
```

Figura 40 Código del simulador - Etiquetas y opciones (5)

En este fragmento se definen las etiquetas textuales del tiempo y los parámetros de rendimiento para los gráficos interactivos. La función de etiquetado lee el valor numérico del eje temporal y lo formatea limitándolo a dos posiciones decimales, acompañado del símbolo de segundos para que sea legible en la interfaz gráfica. La función de opciones establece las directivas de animación para la librería gráfica, indicando que las transiciones entre fotogramas deben ocurrir de forma inmediata y sin retrasos, forzando el redibujado de los lienzos. Al final, se configuran las variables globales

de visualización, asignando el rango dinámico de colores calculado previamente, seleccionando la paleta cromática Jet y activando el parámetro de adaptabilidad para que los gráficos se ajusten automáticamente al tamaño de cualquier pantalla.

Tras pasar de aquí, hay tres condicionales y un `else` donde se comprueban los valores de entrada para saber en qué tipo de gráfica se está: si la profundidad es mayor que 1, estamos ante una gráfica de 3 dimensiones o más; si estamos ante una profundidad igual a 1 pero el número de columnas es más de 1, estamos ante una de 2 dimensiones; y si se descartan las condiciones anteriores, estamos ante 1 dimensión.

```
if (depths > 1) {
    setDisplay(linealChartEl, true);

    function obtenerIntensidadesEnT(t_idx) {
        const intensidades = [];
        for (let r = 0; r < rows; r++) {
            for (let c = 0; c < cols; c++) {
                for (let d = 0; d < depths; d++) {
                    intensidades.push(getConcentration(r, c, d, t_idx));
                }
            }
        }
        return intensidades;
    }

    function obtenerPlano2D(t_idx, z_fijo) {
        const x_2d = [];
        const y_2d = [];
        const c_2d = [];

        for (let r = 0; r < rows; r++) {
            for (let c = 0; c < cols; c++) {
                x_2d.push(c);
                y_2d.push(r);      You, 14 hours ago • Uncommitted changes
                c_2d.push(getConcentration(r, c, z_fijo, t_idx));
            }
        }

        return { x: x_2d, y: y_2d, c: c_2d };
    }

    function obtenerMatrizHeatmap(t_idx, z_fijo) {
        const matriz = [];
        for (let r = 0; r < rows; r++) {
            const fila = [];
            for (let c = 0; c < cols; c++) {
                fila.push(getConcentration(r, c, z_fijo, t_idx));
            }
            matriz.push(fila);
        }
        return matriz;
```

Figura 41 Código del simulador - Visualización 3D (6)

```
const trazas3D = [];
for (let r = 0; r < rows; r++) {
    for (let c = 0; c < cols; c++) {
        for (let d = 0; d < depths; d++) {
            trazas3D.push({
                x: t_axis,
                y: getSeries(r, c, d),
                type: "scatter",
                mode: "lines",
                name: `Fila ${r + 1}, Col ${c + 1}, Prof ${d + 1}`
            });
        }
    }
}
```

Figura 42 Código del simulador - Visualización 3D (7)

Este bloque se ejecuta exclusivamente cuando la simulación posee múltiples capas de profundidad, y su tarea es estructurar los cimientos matemáticos de la vista tridimensional. Primero, reconfigura la interfaz encendiendo el gráfico de curvas temporales y declara tres funciones específicas para realizar cirugías de datos en el espacio volumétrico. La primera extrae en un vector lineal todas las concentraciones de un instante dado; la segunda realiza un corte bidimensional plano en una altura fija del cubo, traduciendo las posiciones a coordenadas cartesianas; y la tercera empaqueta esas mismas concentraciones en una matriz pura, indispensable para mapas de calor. Para finalizar este segmento, el sistema ejecuta un bucle anidado que genera una traza lineal individual para cada compartimento tridimensional, vinculando el eje del tiempo con su curva histórica de concentración.



```
        Plotly.newPlot("lineal_chart", trazas3D, {
            title: "Evolucion temporal de C de los compartimentos",
            xaxis: { title: "Tiempo (s)" },
            yaxis: { title: "Concentracion (C)" },
            hovermode: "closest"
        }, plotConfig);

        const z_central = Math.floor(depths / 2);
        const datosPlanoInicial = obtenerPlano2D(0, z_central);
        const x_coords = [];
        const y_coords = [];
        const z_coords = [];

        for (let r = 0; r < rows; r++) {
            for (let c = 0; c < cols; c++) {
                for (let d = 0; d < depths; d++) {
                    x_coords.push(c);
                    y_coords.push(r);
                    z_coords.push(d);
                }
            }
        }

        const scatterLayout = {
            title: `Corte 2D del cubo (puntos - Z: ${z_central})`,
            xaxis: { title: "Eje X" },
            yaxis: { title: "Eje Y", autorange: "reversed" }
        };

        const heatmapLayout = {
            title: `Mapa de calor de concentracion (Z: ${z_central})`,
            xaxis: { title: "Columnas (X)" },
            yaxis: { title: "Filas (Y)", autorange: "reversed" }
        };
```

Figura 43 Código del simulador - Visualización 3D (8)

```
const volumeLayout = {
    title: "Cubo 3D de concentracion",
    sliders: [{ active: 0, currentvalue: { prefix: "Tiempo: " }, pad: { t: 30 }, steps: [] }],
    updatemenus: [{
        type: "buttons",
        showactive: false,
        x: 0,
        y: 1.12,
        buttons: [{
            label: "Play",
            method: "animate",
            args: [null, animationOptions(80)]
        }, {
            label: "Pause",
            method: "animate",
            args: [[null], animationOptions(0)]
        }]
    }],
    scene: {
        xaxis: { title: "X" },
        yaxis: { title: "Y" },
        zaxis: { title: "Z" }
    }
};
```

Figura 44 Código del simulador - Visualización 3D (9)

En este segmento, el sistema dibuja la primera iteración fija de los gráficos en la pantalla y define los entornos de diseño y controles interactivos para la simulación volumétrica. Inicializa el gráfico de evolución temporal y calcula una coordenada de profundidad central para realizar cortes analíticos en el medio del cubo geométrico. A continuación, genera las grillas de posiciones espaciales mapeando cada coordenada física de los compartimentos y define las configuraciones

de diseño para el gráfico de dispersión bidimensional y el mapa de calor, invirtiendo el sentido del eje vertical para respetar la orientación estándar de lectura de matrices. Por último, diseña el entorno del gráfico volumétrico tridimensional, incorporando componentes de interfaz interactivos como menús desplegables, barras deslizadoras de tiempo y botones automatizados de reproducción y pausa.

```
for (let t = 0; t < pasos_t; t++) {
    const label_t = labelTime(t);
    const datosPlanoT = obtenerPlano2D(t, z_central);

    framesCubo.push({
        name: label_t,
        data: [{
            x: x_coords,
            y: y_coords,
            z: z_coords,
            type: "scatter3d",
            mode: "markers",
            marker: {
                size: 8,
                color: obtenerIntensidadesEnT(t),
                colorscale: colorScale,
                cmin: colorRange.min,
                cmax: colorRange.max,
                showscale: true
            }
        }]
    });

    framesScatter.push({
        name: label_t,
        data: [{
            x: datosPlanoT.x,
            y: datosPlanoT.y,
            type: "scatter",
            mode: "markers",
            marker: {
                size: 12,
                color: datosPlanoT.c,
                colorscale: colorScale,
                cmin: colorRange.min,
                cmax: colorRange.max,
                showscale: true
            }
        }]
    });

    framesHeatmap.push({
        name: label_t,
```

Figura 45 Código del simulador - Animaciones 3D (10)

```
        framesHeatmap.push({
            name: label_t,
            data: [{
                z: obtenerMatrizHeatmap(t, z_central),
                type: "heatmap",
                colorscale: colorScale,
                zmin: colorRange.min,
                zmax: colorRange.max,
                showscale: true
            }]
        });

        sliderSteps.push({
            label: label_t,
            method: "animate",
            args: [[label_t], animationOptions(50)]
        });
    }

    volumeLayout.sliders[0].steps = sliderSteps;
```

Figura 46 Código del simulador - Animaciones 3D (11)

Este fragmento construye la biblioteca completa de animaciones temporales para los tres visores tridimensionales de forma simultánea. El código ejecuta un ciclo que recorre cada instante de tiempo de la simulación y genera en cada iteración un fotograma independiente para cada tipo de vista, asociándole su marca temporal específica. Para el cubo volumétrico, encapsula esferas tridimensionales coloreadas dinámicamente según la concentración del momento; para el visor de dispersión, almacena puntos planos del corte intermedio; y para el mapa de calor, guarda matrices

numéricas actualizadas. Al mismo tiempo, compila los pasos lógicos que se inyectan directamente en la barra de control temporal del layout, permitiendo que el usuario pueda saltar manualmente a cualquier segundo de la simulación.

```
    Promise.all([
        Plotly.newPlot("scatter_chart", [{
            x: datosPlanoInicial.x,
            y: datosPlanoInicial.y,
            mode: "markers",
            type: "scatter",
            marker: {
                size: 12,
                color: datosPlanoInicial.c,
                colorscale: colorScale,
                cmin: colorRange.min,
                cmax: colorRange.max,
                showscale: true
            }
        }], scatterLayout, plotConfig),
        Plotly.newPlot("heatmap_chart", [{
            z: obtenerMatrizHeatmap(0, z_central),
            type: "heatmap",
            colorscale: colorScale,
            zmin: colorRange.min,
            zmax: colorRange.max,
            showscale: true
        }], heatmapLayout, plotConfig),
        Plotly.newPlot("volume_chart", [{
            x: x_coords,
            y: y_coords,
            z: z_coords,
            type: "scatter3d",
            mode: "markers",
            marker: {
                size: 8,
                color: obtenerIntensidadesEnT(0),
                colorscale: colorScale,
                cmin: colorRange.min,
                cmax: colorRange.max,
                showscale: true
            }
        }], volumeLayout, plotConfig)
    ]).then(function() {
        Plotly.addFrames("volume_chart", framesCubo);
        Plotly.addFrames("scatter_chart", framesScatter);
```

Figura 47 Código del simulador - Sincronización de vistas (12)

```
    Plotly.addFrames("heatmap_chart", framesHeatmap);

    volumeChartEl.on("plotly_sliderchange", function(e) {
        const pasoActual = e.step.label;
        Plotly.animate("scatter_chart", [pasoActual], animationOptions(0));
        Plotly.animate("heatmap_chart", [pasoActual], animationOptions(0));
    });

    volumeChartEl.on("plotly_animated", function(e) {
        const frameName = e?.frame?.name;
        if (!frameName) return;
        Plotly.animate("scatter_chart", [frameName], animationOptions(0));
        Plotly.animate("heatmap_chart", [frameName], animationOptions(0));
    });
});
```

Figura 48 Código del simulador - Sincronización de vistas (13)

Esta sección consolida la carga inicial de los visores avanzados y establece un sistema de orquestación de eventos para garantizar que todos los gráficos se muevan al mismo tiempo. Mediante una promesa colectiva, el script renderiza los estados iniciales de dispersión, mapas de calor y del cubo tridimensional en el tiempo cero. Una vez dibujados los lienzos, inyecta las colecciones de fotogramas animados dentro de sus respectivos identificadores visuales y activa dos escuchadores de eventos sobre el gráfico volumétrico principal. Estos interceptan tanto las interacciones manuales del usuario sobre el deslizador como los cambios automáticos producidos por el botón de reproducción, capturando el instante de tiempo en curso y replicando una orden de animación

forzada hacia los gráficos bidimensionales, logrando una sincronía perfecta entre todas las vistas del sistema.



```
} else if (cols > 1) {
    setDisplay(linealChartEl, true);
    setDisplay(volumeChartEl, false);

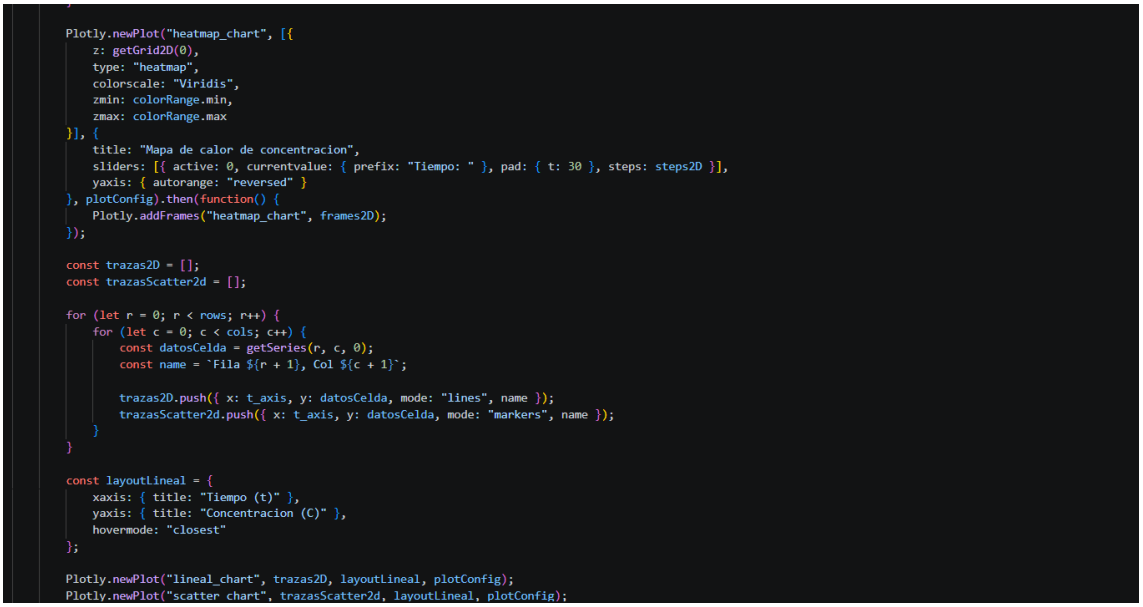
    function getGrid2D(t_idx) {
        const grid = [];
        for (let r = 0; r < rows; r++) {
            const fila = [];
            for (let c = 0; c < cols; c++) {
                fila.push(getConcentration(r, c, 0, t_idx));
            }
            grid.push(fila);
        }
        return grid;
    }

    const frames2D = [];
    const steps2D = [];

    for (let t = 0; t < pasos_t; t++) {
        const lbl = labelTime(t);
        frames2D.push({
            name: lbl,
            data: [{
                z: getGrid2D(t),
                type: "heatmap",
                colorscale: "Viridis",
                zmin: colorRange.min,
                zmax: colorRange.max
            }]
        });
        steps2D.push({
            label: lbl,
            method: "animate",
            args: [[lbl], animationOptions(50)]
        });
    }
```

Figura 49 Código del simulador - Visualización 2D (14)

Este bloque alternativo entra en acción si la simulación no tiene profundidad pero cuenta con una distribución bidimensional compuesta por múltiples columnas. El script adapta el entorno, apagando por completo el lienzo del cubo tridimensional y activando el panel lineal. Define una función interna diseñada para estructurar rejillas de datos planos, extrayendo las concentraciones de cada combinación de fila y columna para un tiempo específico. Posteriormente, inicia un ciclo de compilación temporal que procesa paso a paso los datos de la simulación para construir fotogramas bidimensionales basados en la paleta de color Viridis, enlazando de forma paralela los componentes de control interactivos que gobernarán la línea de tiempo del gráfico.



```
    Plotly.newPlot("heatmap_chart", [{
        z: getGrid2D(0),
        type: "heatmap",
        colorscale: "Viridis",
        zmin: colorRange.min,
        zmax: colorRange.max
    }], {
        title: "Mapa de calor de concentracion",
        sliders: [{ active: 0, currentvalue: { prefix: "Tiempo: " }, pad: { t: 30 }, steps: steps2D }],
        yaxis: { autorange: "reversed" }
    }, plotConfig).then(function() {
        Plotly.addFrames("heatmap_chart", frames2D);
    });

    const trazas2D = [];
    const trazasScatter2d = [];

    for (let r = 0; r < rows; r++) {
        for (let c = 0; c < cols; c++) {
            const datosCelda = getSeries(r, c, 0);
            const name = `Fila ${r + 1}, Col ${c + 1}`;

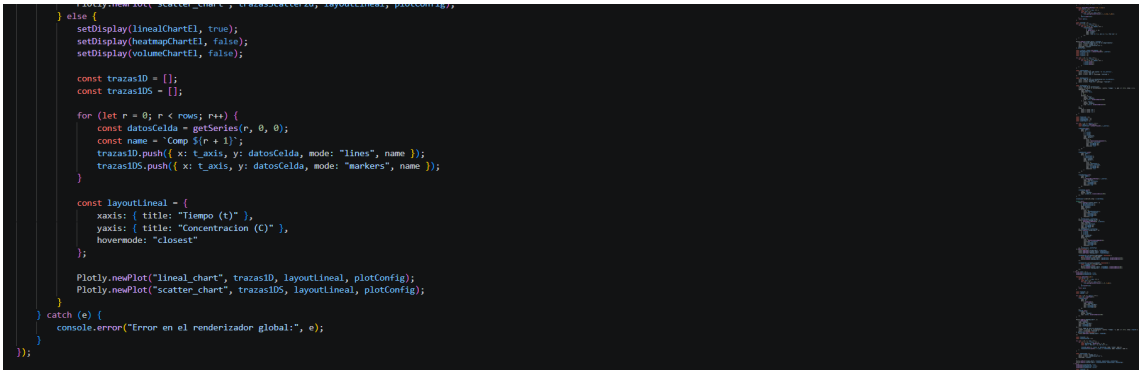
            trazas2D.push({ x: t_axis, y: datosCelda, mode: "lines", name });
            trazasScatter2d.push({ x: t_axis, y: datosCelda, mode: "markers", name });
        }
    }

    const layoutLineal = {
        xaxis: { title: "Tiempo (t)" },
        yaxis: { title: "Concentracion (C)" },
        hovermode: "closest"
    };

    Plotly.newPlot("lineal_chart", trazas2D, layoutLineal, plotConfig);
    Plotly.newPlot("scatter_chart", trazasScatter2d, layoutLineal, plotConfig);
```

Figura 50 Código del simulador - Visualización 2D (15)

Aquí, el script materializa las vistas bidimensionales finales, dibujando el mapa de calor interactivo en su estado inicial e inyectándole su respectiva librería de fotogramas animados al concluir el renderizado. De forma paralela, el sistema ejecuta ciclos de lectura que recorren las coordenadas de la malla plana para obtener las series históricas de concentración de cada intersección espacial, construyendo dos colecciones de trazas independientes: una basada en líneas continuas y otra enfocada en puntos de dispersión discretos. Finalmente, establece un diseño uniforme para los ejes cartesianos, asignando los rótulos de tiempo y concentración, y genera en la pantalla ambas gráficas, permitiendo al usuario realizar comparativas analíticas directas sobre la evolución del plano.



```
    } else {
        setDisplay(linealChartEl, true);
        setDisplay(heatmapChartEl, false);
        setDisplay(volumeChartEl, false);

        const trazas1D = [];
        const trazas1DS = [];

        for (let r = 0; r < rows; r++) {
            const datosCelda = getSeries(r, 0, 0);
            const name = `Comp ${r + 1}`;
            trazas1D.push({ x: t_axis, y: datosCelda, mode: "lines", name });
            trazas1DS.push({ x: t_axis, y: datosCelda, mode: "markers", name });
        }

        const layoutLineal = {
            xaxis: { title: "Tiempo (t)" },
            yaxis: { title: "Concentracion (C)" },
            hovermode: "closest"
        };

        Plotly.newPlot("lineal_chart", trazas1D, layoutLineal, plotConfig);
        Plotly.newPlot("scatter_chart", trazas1DS, layoutLineal, plotConfig);
    }
} catch (e) {
    console.error("Error en el renderizador global:", e);
}
});
```

Figura 51 Código del simulador - Visualización 1D (16)

Este último segmento define la lógica de contingencia para simulaciones lineales simples que poseen una única dimensión. El código apaga de forma definitiva los contenedores de mapas de calor y volúmenes tridimensionales, adaptando la pantalla para mostrar únicamente curvas cartesianas sencillas. Recorre las filas disponibles para capturar los datos históricos de cada compartimento lineal, empaquetándolos tanto en trazas de líneas como de marcadores independientes. Dibuja estas curvas básicas utilizando el diseño estándar y concluye cerrando toda la estructura del script dentro de un bloque de contingencia general que captura cualquier fallo operativo inesperado del renderizado, reportándolo directamente a la consola interna para asegurar que la interfaz de usuario no sufra bloqueos críticos.

8. Resultados del simulador

En este capítulo se mostrará la visualización del dashboard donde están las gráficas y los datos de entradas de los tres modelos dados del NO.

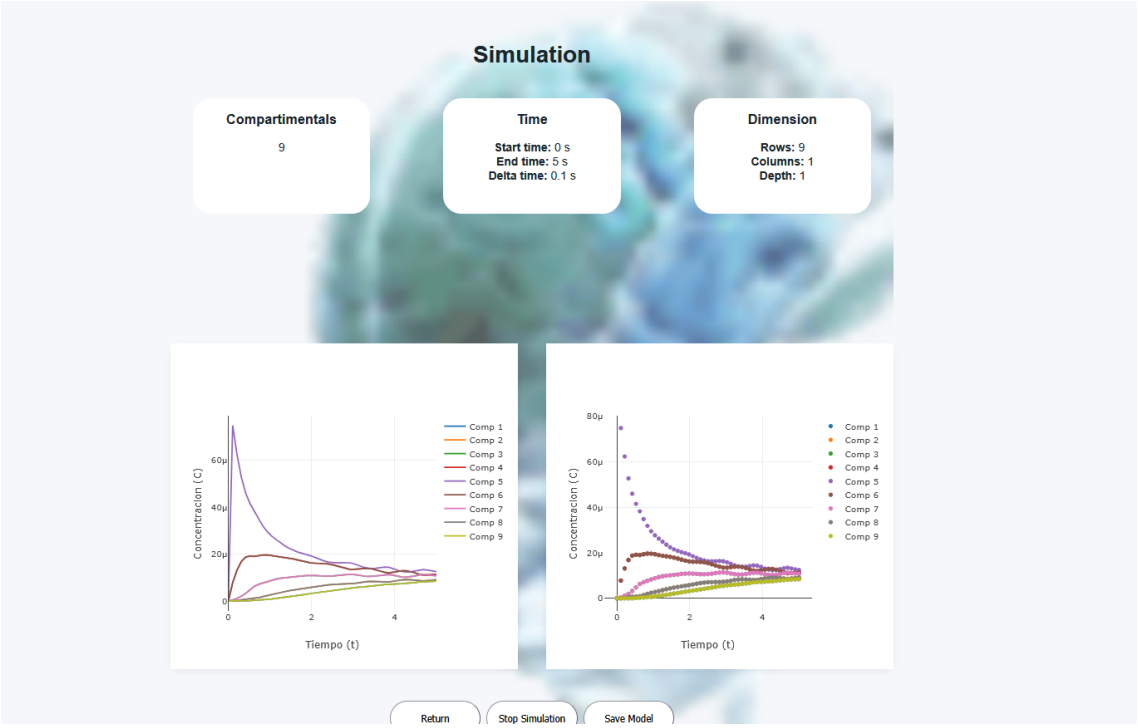


Figura 52 Resultados de la simulación - Vista general

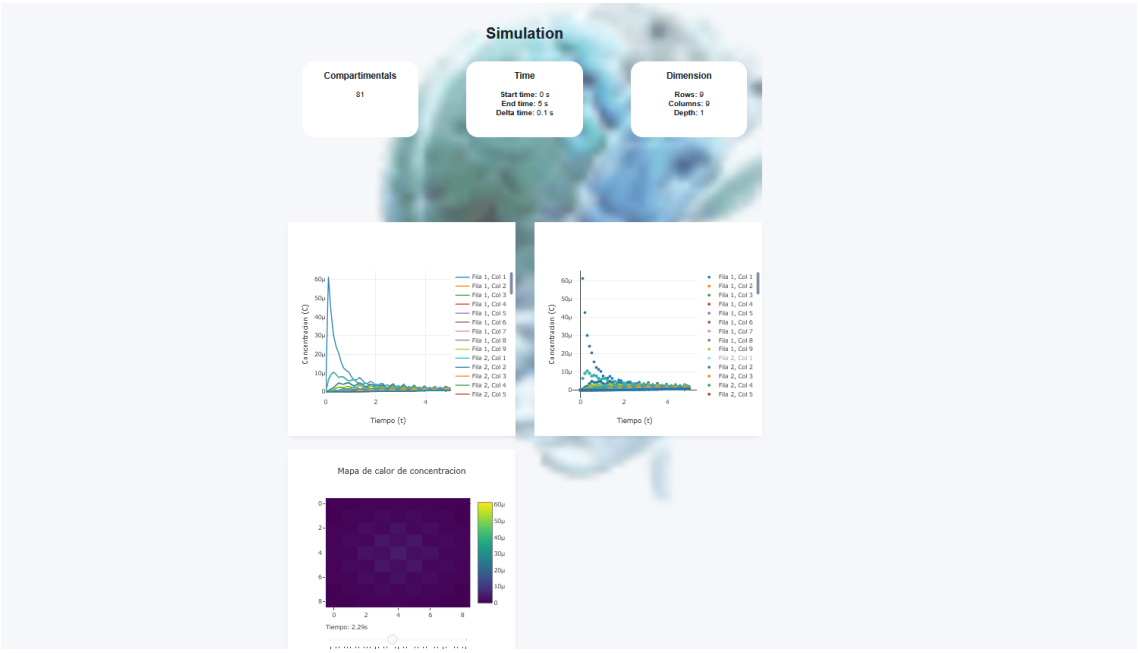


Figura 53 Resultados de la simulación - Gráficas de evolución temporal

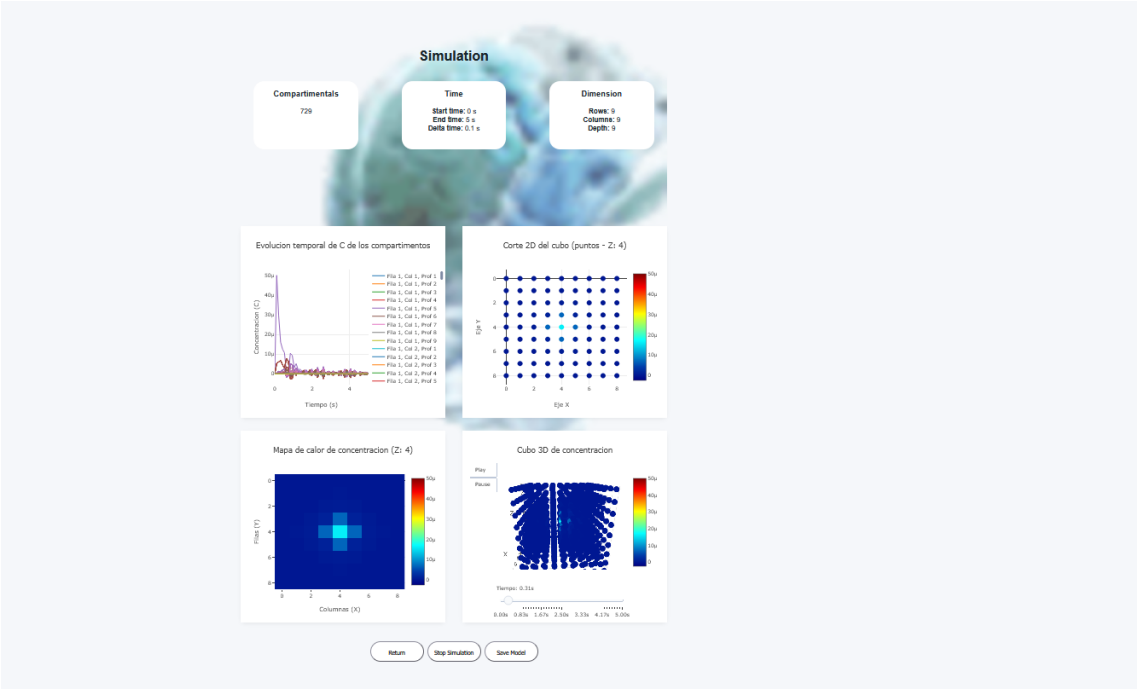


Figura 54 Resultados de la simulación - Visualización tridimensional

9. Conclusiones y trabajo futuro

En este capítulo se aportan las conclusiones del proyecto y las posibles consideraciones de trabajo futuro para la plataforma web de simulación y visualización de modelos de difusión del NO (óxido nítrico), así como el empleo de la IA para mejorar la productividad durante el desarrollo del TFG.

9.1. Conclusiones

El estudio de la difusión del óxido nítrico ha pasado de usar fórmulas matemáticas simples a emplear herramientas muy avanzadas. Hoy en día podemos simular cómo se comporta esta molécula en el cerebro, teniendo en cuenta la forma de las neuronas y cómo aprenden las redes neuronales. Todo esto ha sido posible porque los científicos quieren entender mejor cómo el NO ayuda a que el cerebro aprenda y recuerde.

Los modelos creados en la Universidad de Las Palmas de Gran Canaria han sido muy importantes para este avance. Estos modelos permiten estudiar el NO de una manera más realista, considerando que el cerebro no es un medio uniforme, sino que tiene zonas con diferentes propiedades. Además, han abierto el camino para que otros investigadores puedan seguir profundizando en este campo.

Actualmente, las investigaciones más recientes están usando modelos matemáticos más complejos para estudiar enfermedades como el Parkinson o el ictus, y también están combinando la dinámica del NO con simulaciones de redes neuronales completas.

La plataforma que se ha desarrollado en este Trabajo de Fin de Grado sigue esta misma línea, pero con un enfoque práctico: facilitar que cualquier investigador o estudiante pueda probar sus propios modelos y ver los resultados de forma sencilla, sin necesidad de ser un experto en programación. De esta manera, se acerca el conocimiento científico a más personas y se fomenta la investigación en neurociencia computacional.

9.2. Trabajo futuro

Lo interesante que se podría hacer en este proyecto sería ajustarlo para todo tipo de personas, no solo para personas ciegas, sino también para otros tipos de discapacidades. También se podría añadir la posibilidad de que el usuario añada un modelo con muchos módulos, lo cual ampliaría bastante el proyecto, ya que actualmente se queda escaso en este aspecto. Además, sería interesante poder incluir modelos de otro tipo de disciplinas dentro del machine learning, como el Deep Learning para redes neuronales profundas, y adaptarlo a otro tipo de lenguajes.

10. BIOGRAFÍA

[1] Drapaca, C.S. (2025) Fractional order models of cerebral nitric oxide biotransport. *Journal of Theoretical Biology*, 598, 111987.

[2] Fernández-López, A., Suárez-Araujo, C.P. and García-Báez, P. (2023) Modeling the dynamics of nitric oxide in biological and artificial systems: A computational framework based on automata networks. *Neurocomputing*, 520, 126–138.

[3] Gann, R.G. and Saffman, P.G. (1998) The role of nitric oxide in learning and memory. *Nature Reviews Neuroscience*, 3(4), 283–292.

[4] Garthwaite, J. (1994) Neural nitric oxide signalling: A new view of an old player. *Trends in Neurosciences*, 17(9), 377–380.

[5] Kavdia, M. (2011) Mathematical and Computational Models of Oxidative and Nitrosative Stress. *Critical Reviews in Biomedical Engineering*, 39(5), 461–472.

[6] Lancaster, J.R. (1997) A tutorial on the diffusibility and reactivity of free nitric oxide. *Nitric Oxide: Biology and Chemistry*, 1(1), 18–30.

[7] Pawar, P.D. and Pardasani, K.R. (2024) Nonlinear fractional-order spatio-temporal model for Ca2+ and NO dynamics in neurons. *Journal of Mathematical Biology*, 88, 45.

[8] Philippides, A., Husbands, P. and O'Shea, M. (2000) Four-dimensional neuronal signaling by nitric oxide: A computational analysis. *Journal of Neuroscience*, 20(3), 1199–1207.

[9] Philippides, A., Ott, S.R., Husbands, P., Lovick, T.A. and O'Shea, M. (2005) Modeling Cooperative Volume Signaling in a Plexus of Nitric Oxide Synthase-Expressing Neurons. *Journal of Neuroscience*, 25(28), 6520–6532.

[10] Regirer, S.A. et al. (2005) Mathematical models of nitric oxide transport in a blood vessel. *Biofizika*, 50(2), 321–328.

[11] Snyder, S.H. (1998) The Nobel Prize in Physiology or Medicine 1998. NobelPrize.org. Available at: https://www.nobelprize.org/prizes/medicine/1998/summary/

[12] Trapani, A.M., Sartori, C.A., Gambosi, B., Pedrocchi, A. and Antonietti, A. (2024) Modeling nitric oxide diffusion and plasticity modulation in cerebellar learning. *APL Bioengineering*, 8(2), 026105.

[13] Trapani, A.M., Sartori, C.A., Gambosi, B., Pedrocchi, A. and Antonietti, A. (2025) NODS: A Python Simulator for Nitric Oxide Diffusion in Cerebellar Learning Networks. *bioRxiv*, 2025.01.15.632247.

[14] Wood, J. and Garthwaite, J. (1994) Models of the diffusional spread of nitric oxide: implications for neural nitric oxide signalling. *Neuropharmacology*, 33, 1235–1244.

[15] ODS - Objetivos de Desarrollo Sostenible. INE. Available at: https://www.ine.es/dyngs/ODS/objetivo.htm?id=4869

[16] Python Documentation: `ast` — Abstract Syntax Trees. Available at: https://docs.python.org/es/3.8/library/ast.html

[17] Python Documentation: `subprocess` — Subprocess management. Available at: https://docs.python.org/es/3.9/library/subprocess.html

[18] Python Wiki: Spanish Language Page. Available at: https://wiki.python.org/moin/SpanishLanguage.html

[19] GitHub Documentation: About GitHub and Git. Available at: https://docs.github.com/es/get-started/start-your-journey/about-github-and-git

[20] Suárez Araujo, C.P., Fernández López, P., García Báez, P. and Simoes da Fonseca, J.L. (2006) A Model of Nitric Oxide Diffusion Based in Compartmental Systems. *International Journal of Computing Anticipatory Systems (IJCAS)*, 18, 172–186.

[21] Syková, E. (1997) The extracellular space in the CNS: its regulation, volume and geometry in normal and pathological neural function. *The Neuroscientist*, 3, 28–41.

[22] Syková, E. (2004) Extrasynaptic Volume Transmission and Diffusion parameters of the Extracellular Space. *Neuroscience*, 129, 861–876.

[23] Schuman, E.M. and Madison, D.V. (1994) Nitric Oxide and Synaptic Function. *Annual Review of Neuroscience*, 17, 153–183.

[24] Garthwaite, J. and Boulton, C.L. (1995) Nitric Oxide Signalling in the Central Nervous System. *Annual Review of Physiology*, 57, 683–706.

[25] Suárez Araujo, C.P. (2000) Study and Reflections on the Functional and Organisational Role of Neuromessenger Nitric Oxide in Learning: An Artificial and Biological Approach. *Computer Anticipatory Systems*, AIP 517, 296–307.

[26] Jacquez, J.A. and Simon, C.P. (1993) Qualitative Theory of Compartmental Systems. *SIAM Review*, 35(1), 43–79.

[27] Vaughn, M.W., Kuo, L. and Liao, J.C. (1998) Effective diffusion distance of nitric oxide in the microcirculation. *American Journal of Physiology*, 1705–1714.

[28] Lancaster, J.R. (1994) Simulation of the diffusion and reaction of endogenously produced nitric oxide. *Proceedings of the National Academy of Sciences USA*, 91, 8137–8141.

[29] Malinski, T., Taha, Z., Grunfeld, S., Patton, S., Kapturczak, M. and Tomboulian, P. (1993) Diffusion of nitric oxide in the aorta wall monitored in situ by porphyrinic microsensors. *Biochemical and Biophysical Research Communications*, 193, 1076–1082.

[30] Balbatun, A., Louka, F.R. and Malinski, T. (2003) Dynamics of nitric oxide release in the cardiovascular system. *Acta Biochimica Polonica*, 50(1), 61–68.

[31] Suárez-Araujo, C.P., Fernández-López, P. and García-Báez, P. (2011) ANDINO: Automata Network Model for the Diffusion of Nitric Oxide in Biological and Artificial Neural Systems. A Preliminary Study. *Proceedings of the 12th International Symposium on Computational Intelligence and Informatics (CINTI)*, 73–78.

[32] Bredt, D.S. and Snyder, S.H. (1994) Nitric Oxide: A Physiologic Messenger Molecule. *Annual Review of Biochemistry*, 63, 175–195.

[33] Moncada, S. et al. (1999) The L-Arginine: Nitric oxide pathway. *Journal of Cardiovascular Pharmacology*, 17, S1–S9.

[34] Huizhong, W. and Mu-ming Poo. (2001) Retrograde signalling at central synapse. *Proceedings of the National Academy of Sciences USA*, 98(20), 11009–11015.

[35] Regidor, J. and Suárez Araujo, C.P. (1995) Retrograde neural Messenger in the Brain. Preliminary Study on the Implications in the Artificial Neural Networks. *Brain Processes, Theories and Models*, MIT Press, 360–369.

[36] Dömösi, P. and Nehaniv, C.L. (2005) Algebraic Theory of Automata Networks. ISBN: 9780898715699.

[37] Suárez Araujo, C.P., Fernández López, P. and García Báez, P. (2011) Nitric Oxide Diffusion Attributes in Biological and Artificial Environments: A Computational Study. *Majlesi Journal of Electrical Engineering*, 5(1), 73–82.

[38] Anderson, D.H. (1983) Compartmental analysis and tracer kinetics. *Springer-Verlag*, Berlin.

[39] Suárez Araujo, C.P., Fernández López, P. and García Báez, P. (2001) Towards a Model of Volume Transmission in Biological and Artificial Neural Networks: A CAST Approach. *Proceedings of EUROCAST 2001*, LNCS 2178, 328–342.

[40] Pichler, F. and Schwaertzel, H. (Eds.) (1992) CAST Methods in Modelling. *Springer Verlag*, Berlin Heidelberg New York. ISBN 3-540-55405-X.

[41] Kohonen, T. (1972) Correlation matrix memories. *IEEE Transactions on Computers*, C-21, 353–359.

[42] Hebb, D.O. (1949) The Organization of Behavior. *Wiley and Sons*, New York.

[43] Crank, J. (1956) The Mathematics of Diffusion. *Oxford Science Publications*.

[44] Fernández López, P. and Suárez Araujo, C.P. (2000) Hacia un Modelo de Difusión del Comportamiento del Óxido Nítrico como Mensajero Celular Retrógrado. *Actas del XVIII Congreso Anual de la Sociedad Española de Bioingeniería*, 165–169. ISBN 84-699-311-3.

[45] Philippides, A., Husbands, P. and O'Shea, M. (1998) Neural Signalling: It's a Gas! *Proceedings of the 8th International Conference on Artificial Neural Networks*, Springer Verlag, 979–984.

[46] Dinerman, J.L., Dawson, T.M., Schell, M.J., Snowman, A. and Snyder, S.H. (1994) Endothelial nitric oxide synthase localised to hippocampal pyramidal cells: Implications for synaptic plasticity. *Proceedings of the National Academy of Sciences USA*, 91, 4214–4218.

[47] Krekelberg, B. (1997) Modelling cortical self-organization by volume learning. *Doctoral Dissertation*, King's College London.

[48] Von Neumann, J. and Burks, A.W. (1966) Theory of self-reproducing automata. *University of Illinois Press*, Urbana.

[49] Hawkins, R.D., Kandel, E.R. and Siegelbaum, S.A. (1993) Learning to modulate transmitter release: themes and variations in synaptic plasticity. *Annual Review of Neuroscience*, 16, 625–665.

[50] Zhuo, M., Laitinen, J.T., Li, X.C. and Hawkins, R.D. (1998) On the respective roles of nitric oxide and carbon monoxide in long-term potentiation in the hippocampus.

[51] Docker Documentation. Docker Inc. Available at: https://docs.docker.com/

[52] Docker Compose Documentation. Docker Inc. Available at: https://docs.docker.com/compose/

[53] Dockerfile Reference. Docker Inc. Available at: https://docs.docker.com/engine/reference/builder/

Learning and Memory, 5, 467–480.

11. Apéndice A: Abreviaturas

Cuadro 12 Abreviaturas utilizadas en el documento

Abreviatura	**Significado**
ADN	Ácido Desoxirribonucleico
API	Application Programming Interface (Interfaz de Programación de Aplicaciones)
ARN	Ácido Ribonucleico
AST	Abstract Syntax Tree (Árbol de Sintaxis Abstracta)
AWS	Amazon Web Services (Servicios Web de Amazon)
cAMP	Adenosin monofosfato cíclico
cGMP	Guanosin monofosfato cíclico
CSS	Cascading Style Sheets (Hojas de Estilo en Cascada)
Django	Framework de desarrollo web para Python
Docker	Plataforma de contenedorización de aplicaciones
Docker Compose	Herramienta para definir y ejecutar aplicaciones Docker multi-contenedor
GET	Método HTTP para solicitar datos de un recurso
Git	Sistema de control de versiones distribuido
GitHub	Plataforma de alojamiento de repositorios Git
HTML	HyperText Markup Language (Lenguaje de Marcado de Hipertexto)
HTTP	HyperText Transfer Protocol (Protocolo de Transferencia de Hipertexto)
IA	Inteligencia Artificial
JavaScript	Lenguaje de programación interpretado para desarrollo web
JSON	JavaScript Object Notation (Notación de Objetos JavaScript)
MVT	Model-View-Template (Modelo-Vista-Plantilla)
MVP	Producto Mínimo Viable (Minimum Viable Product)
NO	Óxido nítrico (Nitric Oxide)
NumPy	Biblioteca de Python para computación numérica
ODS	Objetivos de Desarrollo Sostenible
ORM	Object-Relational Mapping (Mapeo Objeto-Relacional)
Pandas	Biblioteca de Python para análisis de datos
pgAdmin	Herramienta de administración para PostgreSQL
Plotly	Biblioteca de Python para visualización de datos interactiva
POST	Método HTTP para enviar datos a un recurso
PostgreSQL	Sistema de gestión de bases de datos relacional objeto
Python	Lenguaje de programación de alto nivel
RAM	Random Access Memory (Memoria de Acceso Aleatorio)
SciPy	Biblioteca de Python para computación científica y técnica
SCRUM	Metodología ágil para la gestión de proyectos

Continúa en la siguiente página

Tabla continua: Abreviaturas	
Abreviatura	**Significado**
Seaborn	Biblioteca de Python para visualización de datos estadísticos
SNC	Sistema Nervioso Central
SNP	Sistema Nervioso Periférico
SQL	Structured Query Language (Lenguaje de Consulta Estructurado)
SVG	Scalable Vector Graphics (Gráficos Vectoriales Escalables)
TFG	Trabajo de Fin de Grado
UI	Interfaz de Usuario (User Interface)
UX	Experiencia de Usuario (User Experience)
VS Code	Visual Studio Code (Editor de código fuente)
WAI-ARIA	Web Accessibility Initiative - Accessible Rich Internet Applications
WCAG	Web Content Accessibility Guidelines (Pautas de Accesibilidad al Contenido Web)
XSS	Cross-Site Scripting (Ataque de secuencias de comandos entre sitios)

12. Apéndice B: Uso de la IA

Se ha empleado la IA con fines de aprendizaje del framework Django, puesto que es la primera vez que se había usado y era necesario tener los conceptos para desarrollar. También se ha usado para agilizar las investigaciones de librerías Python para la automatización de comandos en diferentes tipos de sistemas operativos, y para mejorar los conocimientos de Docker y Docker-Compose con el objetivo de integrarlo en el proyecto. Además, se ha usado para mejorar la legibilidad de la memoria y, por último, para generar SVG para los diferentes tipos de iconos de la plataforma web.

13. Apéndice C: Estado del arte del modelado del óxido nítrico

13.1. Evolución de los modelos (1994-2026)

La evolución de los modelos de difusión del NO se puede dividir en varias etapas, cada una con avances significativos en la comprensión de esta molécula:

Etapa	Años	Características	Autores
Primeros modelos analíticos	1994-1998	Ecuaciones de difusión (Fick), fuentes puntuales, geometría simple	Wood y Garthwaite [14], Lancaster [28]
Modelos anatómicos	2000-2005	Incorporan morfología realista de neuronas y sumideros de NO	Philippides et al. [8, 9]
Modelos compartimentales	2001-2011	Espacio discreto, permiten no-homogeneidad y anisotropía	Suárez Araujo y Fernández López [39, 20]
Modelos de autómatas (ANDINO)	2011	Redes de autómatas, visión molecular del NO	Suárez-Araujo et al. [31]
Modelos fraccionarios	2024-2025	Capturan difusión anómala y memoria celular	Pawar y Pardasani [7], Drapaca [1]
Simuladores integrados (NODS)	2024-2025	Simulación en redes neuronales completas (cerebelo)	Trapani et al. [12, 13]

Cuadro 13 Evolución de los modelos de difusión del NO

13.2. Modelos clave y sus aportaciones

13.2.1. Modelos analíticos (1994-1998)

Los primeros modelos computacionales se centraron en caracterizar las propiedades de señalización volumétrica del NO mediante ecuaciones de difusión basadas en las leyes de Fick.

Wood y Garthwaite (1994) [14] desarrollaron un modelo de difusión de NO en medio isotrópico, calculando su radio de acción en aproximadamente 200 micrómetros. Este trabajo sentó las bases para entender que el NO no actúa de forma puntual como un neurotransmisor clásico, sino que genera ”nubes difusas”que afectan a volúmenes de tejido que contienen múltiples neuronas y sinapsis.

Lancaster (1994) [28] propuso una simulación de difusión-reacción del NO que considera su vida media y las reacciones químicas que lo inactivan. Estos modelos presentaban limitaciones importantes, como la consideración de fuentes puntuales y medios homogéneos, lo que no refleja la realidad biológica del tejido neural.

13.2.2. Modelos anatómicos (2000-2005)

Los modelos basados en la estructura anatómica real de los sistemas nitrérgicos supusieron un avance significativo. **Philippides y colaboradores (2000)** [8] desarrollaron un modelo 4D de señalización neuronal por NO que considera la geometría realista de las fuentes de NO, demostrando que la morfología de las fuentes influye de manera crucial en la dinámica espaciotemporal de la señal.

Posteriormente, **Philippides et al. (2005)** [9] investigaron los plexos de fibras ultrafinas que expresan óxido nítrico sintasa (NOS). Su modelado reveló que, aunque una fibra individual de pequeño diámetro genera una señal limitada, la cooperación entre muchas fibras puede generar una señal volumétrica extensa y homogénea en espacio y tiempo.

13.2.3. Modelos compartimentales (2001-2011)

El grupo de investigación del Instituto de Cibernética de la Universidad de Las Palmas de Gran Canaria, liderado por Suárez Araujo y Fernández López, desarrolló una línea de investigación pionera en el modelado de la difusión del NO.

Suárez Araujo y Fernández López (2001) [39] propusieron un modelo basado en compartimentos y en cómo se mueven las sustancias en el espacio, usando el método CAST (*Computer Aided Systems Theory*) para su desarrollo. Este modelo permite tener en cuenta que el entorno del cerebro no es uniforme ni igual en todas las direcciones, y también permite usar formas de producción de NO que se parecen más a lo que ocurre realmente en el cerebro

13.2.4. Modelo ANDINO: redes de autómatas (2011)

Suárez-Araujo et al. (2011) [31] presentaron **ANDINO** (*Automata Network Model for the Diffusion of Nitric Oxide*), un modelo basado en redes de autómatas con conexiones hexagonales. Cada autómata representa una molécula de NO, permitiendo estudiar la dinámica de difusión desde una perspectiva molecular.

El modelo incorpora dos tipos de autorregulación: por nivel de NO y por tiempo de vida media. Los resultados mostraron que la formación de agrupamientos moleculares y estructuras complejas depende de la capacidad de expansión de las moléculas de NO, con implicaciones en la transmisión de información en redes neuronales biológicas y artificiales.

13.2.5. Modelos de difusión anómala y orden fraccional (2024-2025)

Una de las tendencias más recientes aborda la difusión anómala del NO, un fenómeno causado por el atrapamiento de la molécula en micropartículas endoteliales circulantes, que se acentúa en enfermedades cerebrovasculares.

Pawar y Pardasani (2024) [7] construyeron un modelo espacio-temporal fraccionario no lineal para Ca^{2+} y NO en neuronas, incorporando ecuaciones de reacción-difusión de orden fraccionario. Este modelo permite explorar las condiciones que conducen a disfunciones en estos sistemas y a enfermedades como el Parkinson.

Drapaca (2025) [1] propuso una ecuación de reacción-difusión fraccionaria espacial para modelar el biotransporte de NO cerebral, demostrando que para una difusión anómala de orden fraccionario, la concentración de NO en el endotelio aumenta con el tiempo, lo que concuerda con estudios sobre el ictus.

13.2.6. Simuladores integrados: NODS (2024-2025)

El avance más significativo de los últimos años es la integración de la dinámica del NO en simuladores de redes neuronales completas. **Trapani y colaboradores (2024-2025)** [12, 13] desarrollaron **NODS** (*NO Diffusion Simulator*), un módulo Python diseñado para modelar la producción y difusión del NO en el marco de una red neuronal espicular cerebelosa.

Las simulaciones del condicionamiento clásico del reflejo de parpadeo demostraron que:

- La difusión del NO afecta significativamente la plasticidad sináptica, ajustando dinámicamente las tasas de aprendizaje según los patrones de actividad sináptica.
- El NO actúa como un mecanismo de **metaplasticidad** que mejora la capacidad del cerebelo para priorizar entradas relevantes y mitigar la interferencia en el aprendizaje.
- El NO funciona como un indicador contextual que optimiza las tasas de aprendizaje para el control motor y la adaptación a nuevas tareas.

Estos hallazgos son consistentes con estudios experimentales que demuestran la necesidad del NO tanto en la potenciación como en la depresión a largo plazo en las sinapsis entre fibras paralelas y células de Purkinje.

13.3. Tendencias actuales y direcciones futuras

El estado del arte actual se caracteriza por las siguientes tendencias:

1. **Modelos multiescala**: Integración desde la dinámica molecular hasta el comportamiento de redes neuronales completas.
2. **Modelos de orden fraccional**: Capturan fenómenos de difusión anómala y efectos de memoria celular asociados a enfermedades neurodegenerativas.
3. **Simuladores accesibles y extensibles**: Herramientas como NODS que facilitan la reproducción y extensión de experimentos computacionales.
4. **Integración con teorías de la información**: Cuantificación de la capacidad de representación de información del sistema de señalización por NO.
5. **Plataformas web accesibles**: Herramientas que permiten ejecutar y visualizar modelos sin necesidad de configuraciones técnicas complejas, como la desarrollada en el presente TFG.